



Figura 13.152 Conjuntos ordenados y desordenados

Para aquellos atributos cuya multiplicidad sea mayor que uno, se puede situar una de las palabras reservadas de ordenación después de la cadena de atributos, entre llaves, formando parte de una cadena de propiedades.

Si se omite la palabra clave de ordenación, entonces el conjunto no está ordenado.

Discusión

Un conjunto ordenado posee información en la ordenación, información que es adicional respecto a las entidades del conjunto en sí. Esto es una información real. Por tanto, no es derivable sino que se tiene que especificar cuándo se añade una entidad. En otras palabras, en toda operación que añada una entidad, es preciso especificar su posición dentro de la lista de entidades. Por supuesto, se puede implementar una operación de tal modo que la nueva entidad se inserte en alguna localización implícita, tal como el comienzo o el final de la lista. Y el hecho de que un conjunto esté ordenado no implica que se vaya a admitir cualquier ordenación de las entidades. Estas decisiones tiene que tomarlas quien haga el modelo. En general, la posición de la nueva entidad dentro de la lista es un parámetro de la operación de creación.

Observe que la ordenación de una asociación binaria tiene que ser especificada independientemente para cada dirección. La ordenación carece de sentido a no ser que la multiplicidad en una dirección sea mayor que uno. Una asociación puede ser completamente desordenada, ordenada en una dirección y no en la otra, o bien puede ser ordenada en ambas direcciones.

Consideremos una asociación entre las clases A y B que está ordenada en la dirección de B. Entonces lo normal será que los nuevos enlaces se añadan como una operación sobre un objeto A, especificando un objeto B y una posición en la lista de objetos B existentes para el nuevo enlace. Con cierta frecuencia, una operación sobre un objeto A crea un nuevo objeto B y también crea un enlace entre A y B. Es preciso añadir la lista a la lista de enlaces que mantiene A. Es posible crear un nuevo enlace desde el lado de B pero generalmente el nuevo enlace se insertará en una posición por defecto de la lista que va de A hasta B, porque la posición dentro de esa lista tiene escaso significado desde el lado de B. Por supuesto, un programador puede implementar situaciones más complicadas si fuera necesario.

Resulta poco frecuente hallar una asociación que esté ordenada en ambas direcciones, porque puede resultar engorroso especificar el punto de inserción en las dos direcciones. Pero es posible, especialmente si se añaden los nuevos enlaces en localizaciones por defecto en ambas direcciones.

Observe que un conjunto ordenado no contiene ninguna información adicional más allá de la información relativa al conjunto de entidades. La ordenación ahorra tiempo en un algoritmo pero no agrega información alguna. Se puede considerar como una optimización de diseño y no

es necesario incluirla en un modelo de análisis. Puede ser especificada como valor de la propiedad de ordenación, pero no requiere que una operación especifique una localización para la nueva entidad añadida al conjunto. La localización de la nueva entidad debe ser determinada automáticamente por el método que examina los atributos según los cuales se ordena la lista.

padre

El elemento más general dentro de una relación de generalización. Para una clase recibe el nombre de superclase. Una cadena de una o más relaciones de padre (esto es, el cierre transitivo) es un antecesor. Lo contrario es un hijo.

Véase generalización.

palabra clave

Se entiende que una palabra clave es un adorno textual que categoriza a un elemento del modelo cuando éste carece de su propia sintaxis distintiva.

Véase también marcador gráfico, estereotipo.

Notación

Las palabras reservadas se emplean para aquellos elementos incorporados del modelo que carecen de su propia notación, así como para los estereotipos definibles por el usuario. La notación general para utilizar una palabra reservada consiste en encerrarla entre comillas («») «palabra clave».

Cuando la palabra clave forma parte de un símbolo de área, tal como un rectángulo de clase, entonces la palabra reservada se pone dentro de los límites del símbolo.

En el texto de este documento se describen algunas palabras claves que se tratan como palabras claves dentro de la notación. Hay otros nombres que están a disposición de los usuarios para que los empleen como nombres de estereotipo. No se permite utilizar un nombre de estereotipo que coincida con una palabra clave predefinida.

Discusión

El número de símbolos visuales fácilmente distinguibles es limitado. Por tanto, la notación UML hace uso de palabras claves de texto para distinguir las variantes de un tema común, entre las que se encuentran las subclases de metamodelo de una clase base, los estereotipos de una clase base de metamodelo y los grupos de elementos de listas. Desde el punto de vista del usuario, la distinción de metamodelo entre subclases de metamodelo y estereotipos no suele ser importante, aunque lo sea, por supuesto, para los constructores de herramientas y para otras personas que implementen el metamodelo.

paquete

Término que denota un mecanismo de propósito general para organizar en grupos los elementos. Los paquetes se pueden anidar dentro de otros paquetes. Un sistema puede corresponder a un único paquete de alto nivel, con todo el resto del modelo contenido recursivamente en él. En un paquete pueden aparecer tanto elementos del modelo como diagramas.

Véase también acceso, dependencia, importar, modelo, espacio de nombres, subsistema.

Semántica

La palabra *paquete* alude a un agrupamiento de elementos y diagramas del modelo. Todo elemento del modelo que no forme parte de otro elemento del modelo deberá estar declarado únicamente en un espacio de nombres; el espacio de nombres que contiene la declaración de un elemento se denomina propietario del elemento. Un paquete es un espacio de nombres de propósito general que puede poseer cualquier clase de elemento de modelo que no esté limitado a una sola clase de propietario. Todo diagrama debe ser poseído exactamente por un paquete, que puede estar anidado dentro de otro paquete (y por tanto será poseído por este último). Los paquetes pueden contener paquetes subordinados y elementos ordinarios del modelo. Algunos paquetes pueden ser subsistemas o modelos. La descripción de todo el sistema puede concebirse como un único paquete de subsistema de alto nivel, que contiene absolutamente todo lo demás. Todas las clases de elementos de modelo de UML se pueden organizar en paquetes.

Los paquetes poseen elementos del modelo, subconjuntos del modelo y diagramas. Los paquetes son la base del control de configuraciones, del almacenamiento y del control de accesos. Todo elemento puede ser poseído directamente por otro elemento del modelo o por un único paquete, de tal modo que la jerarquía de posesión es un árbol estricto. Sin embargo, los elementos del modelo (incluyendo los paquetes) pueden hacer referencia a otros elementos de otros paquetes, así que la red de utilización es un grafo.

Las clases especiales de paquete son el modelo, el subsistema y el sistema. Un sistema denota un subsistema que es la raíz de la jerarquía del paquete. Se trata del único elemento del modelo que no es poseído por algún otro elemento del modelo. Indirectamente, incluye todo lo que hay en el modelo. Existen varios estereotipos predefinidos de modelo y de subsistema.

Véase el Capítulo 14, Elementos Estándar, para más detalles.

Los paquetes pueden tener relaciones de dependencia con otros paquetes. En la mayoría de los casos, estas dependencias no son otra cosa que un resumen de las dependencias que existen entre los contenidos de los paquetes. Un uso de dependencia entre dos paquetes significa que existe al menos una dependencia de uso entre elementos de los dos paquetes (esto no quiere decir que toda pareja de elementos tenga esa dependencia).

La dependencia de acceso es propia de los paquetes en sí y no es un compendio de dependencias de sus elementos. Indica que a los elementos del paquete cliente se les otorga permiso para tener relaciones con los elementos del paquete proveedor. Las relaciones también están sometidas a especificaciones de visibilidad. El acceso no significa que los nombres de los elementos que se encuentran en el paquete blanco ocupen el espacio de nombres del paquete origen: los espacios de nombres son diferentes y los elementos pueden identificarse de modo

único mediante nombres de ruta que incluyen a los paquetes anidados. La importancia de la variación de dependencia de acceso es como la sentencia **uses** de Ada. Añade los nombres del espacio de nombres del proveedor al espacio de nombres del cliente (no deben existir conflictos). Pero la dependencia de acceso no altera el espacio de nombres del cliente. Se trata, principalmente, de un mecanismo de control de acceso en los proyectos de desarrollo más extensos y no una relación semántica fundamental.

Un paquete anidado tiene acceso a todos los elementos que estén contenidos directamente en paquetes externos (hasta cualquier nivel de anidamiento) sin necesidad de dependencias de importación ni de visibilidades. Sin embargo, el paquete tiene que importar los paquetes que contenga para poder ver su interior. En general, un paquete contenido es un límite de encapsulamiento.

Los paquetes definen la visibilidad de los elementos que contienen como **privada**, **protegida** o **pública**. Los elementos privados no están disponibles de ningún modo fuera del paquete que los contiene (independientemente de las importaciones). Los elementos protegidos sólo están disponibles para aquellos paquetes que tengan generalizaciones al paquete que los contiene y los elementos públicos están disponibles para los paquetes que importan y para los descendientes del paquete.

Véase acceso para una descripción completa de las reglas de visibilidad que se aplican a los elementos de los distintos paquetes.

Notación

Los paquetes se representan mediante un rectángulo grande con un rectángulo pequeño (una “ficha”) asociado a una esquina (normalmente, el lado izquierdo de la parte superior del rectángulo grande). Se trata del icono de una carpeta.

Si no se muestra el contenido del paquete, entonces el nombre del paquete se sitúa dentro del rectángulo grande. Si se muestra el contenido del paquete, entonces el nombre del paquete se puede situar dentro de la ficha.

Se puede situar la cadena de una palabra reservada por encima del nombre del paquete. Entre las palabras reservadas se cuentan subsistema, sistema y modelo. Los estereotipos definidos por el usuario se anotan también con palabras reservadas pero no deben entrar en conflicto con las palabras reservadas predefinidas.

Se puede poner entre llaves una lista de propiedades detrás o por debajo del nombre del paquete. Ejemplo: {**abstract**}.

El contenido del paquete se puede mostrar dentro del rectángulo grande.

La visibilidad de un elemento de paquete fuera del paquete se puede indicar anteponiendo al nombre del elemento un símbolo de visibilidad (‘+’ denota pública, ‘-’ significa privada y ‘#’ quiere decir protegida).

Se pueden dibujar las relaciones entre símbolos de paquete para mostrar relaciones que existan al menos entre algunos de los elementos de los paquetes. En particular, la dependencia de otros paquetes (salvo las dependencias de permiso, tales como el acceso y la importación) implican que existe una o más dependencias entre los elementos).

Opciones de representación

Las herramientas también pueden mostrar la visibilidad visualizando selectivamente aquellos elementos que satisfacen un determinado nivel de visibilidad; por ejemplo, mostrando únicamente los elementos públicos.

Las herramientas pueden mostrar la visibilidad empleando algún marcador gráfico, tal como un color o un tipo de letra.

Reglas de estilo

Se supone que los paquetes que posean contenidos extensos se mostrarán como sencillo iconos con nombre, en los cuales se puede acceder dinámicamente al contenido “haciendo zoom” para llegar a una visualización detallada.

Ejemplo

La Figura 13.153 muestra la estructura de paquete de un subsistema para el procesamiento de pedidos. El subsistema en sí se representa mediante un paquete con un estereotipo. Contiene varios paquetes ordinarios. Las dependencias entre paquetes se muestran mediante flechas discontinuas. La figura muestra también algunos paquetes externos de los cuales depende el subsistema. Éstos pueden ser componentes estandarizados o bien elementos de una biblioteca.

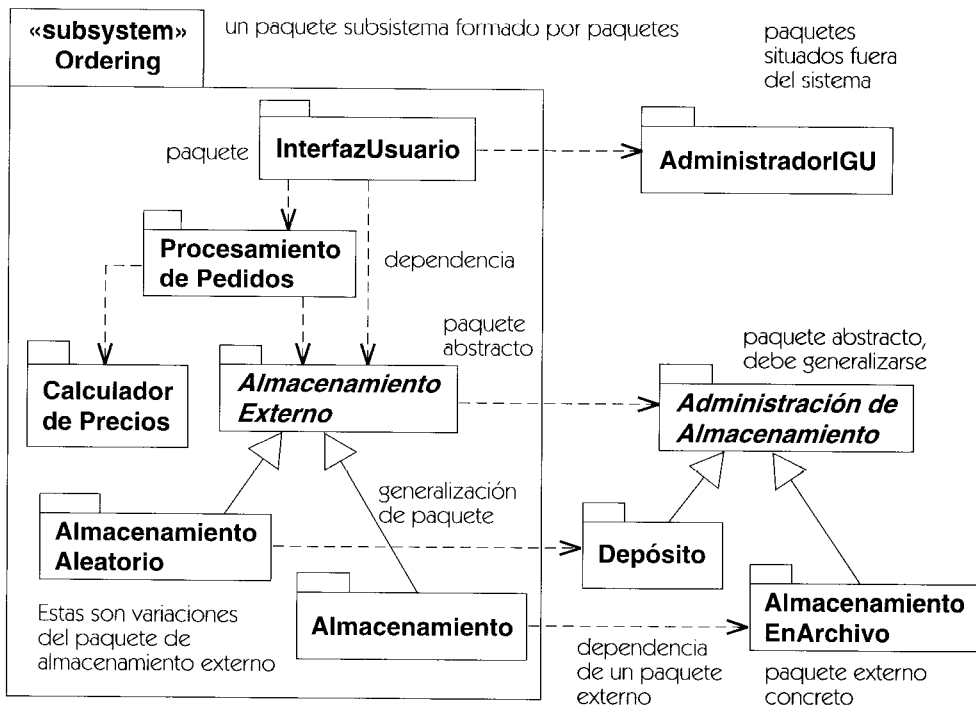


Figura 13.153 Los paquetes y sus relaciones

La generalización entre paquetes muestra las variaciones de un paquete genérico. Por ejemplo, el paquete **AlmacenamientoExterno** se puede implementar como un **AlmacenamientoAleatorio** o un **Almacenamiento**.

Discusión

Los paquetes están destinados primordialmente a ser mecanismos de control de acceso y configuración que permitan a los desarrolladores, especialmente los que trabajan en grandes grupos, organizar grandes modelos y hacerlos evolucionar sin estorbarse unos a otros. Significan, inherentemente, lo que los desarrolladores quieran que signifiquen. Mas prácticamente, los paquetes deberían seguir algún tipo de límite semántico para que puedan ser útiles. Como están destinados a ser unidades de control de configuraciones, deberían contener elementos que tengan probabilidades de evolucionar al unísono. Los paquetes también sirven para agrupar aquellos elementos que deben ser compilados a la vez. Si un cambio de uno de los elementos impone la recompilación de otros elementos, entonces quizá convenga poner a ambos en un mismo paquete.

Todo elemento del modelo debe ser poseído exactamente por un paquete u otro elemento del modelo. En caso contrario, el mantenimiento del modelo, las versiones y el control de configuración se vuelven imposibles. El paquete que posee un elemento del modelo controla su definición. Se puede hacer referencia a él y es posible utilizarlo en otros paquetes, pero hacer un cambio en él requiere tener un permiso de acceso y el derecho de actualización para el paquete que lo posee.

Elementos estándar

access, extend, facade, framework, stub, system.

parámetro

Término que denota la especificación de una variable que se puede modificar, pasar o proporcionar. Los parámetros pueden tener nombre, tipo y dirección. Se utilizan en operaciones, mensajes, eventos y plantillas. Contrastar con: argumento.

Las dependencias de uso de parámetro relacionan a una operación que posee un parámetro o a una clase que posea esa operación con la clase del parámetro.

Véase también argumento, enlace.

Semántica

Un parámetro es una reserva de espacio para un argumento, que se enlaza con él cuando se hace uso del elemento que lo contiene. Los parámetros restringen los valores que puede admitir el argumento. Tienen las partes siguientes:

valor por defecto	Es una expresión que proporciona el valor que se utilizará si no se proporciona un argumento para el parámetro. La expresión se evalúa cuando la lista de parámetros se enlaza con los argumentos.
dirección	Es la dirección del flujo de información del parámetro; se trata de una enumeración con los valores siguientes:
in	Un parámetro de entrada pasado por valor. Los cambios efectuados en el parámetro no están disponibles para el emisor.
out	Un parámetro de salida. No existe valor de entrada. El valor final está disponible para el emisor.
inout	Un parámetro de entrada que se puede modificar. El resultado final está disponible para el emisor.
return	Valor proporcionado por una llamada. El valor está disponible para el emisor. Semánticamente, no tiene diferencias con respecto a un parámetro out, pero el resultado está disponible para su utilización en una expresión inline.

Las opciones precedentes pueden no estar disponibles directamente en todos los lenguajes de programación, pero el concepto que subyace a todas ellas tiene sentido en la mayoría de los lenguajes y se puede hacer corresponder con una implementación razonable.

nombre	Es el nombre del parámetro. Tiene que ser único dentro de su lista de parámetros.
tipo	Se trata de una referencia de un clasificador (una clase, tipo de datos o interfaz en la mayoría de los procedimientos). El argumento que esté enlazado con el parámetro tiene que ser una instancia del clasificador o de uno de sus descendientes.

Notación

Todo parámetro se representa mediante una cadena de texto que se puede analizar sintácticamente para descomponerla en las distintas propiedades del parámetro. La sintaxis por defecto es:

dirección nombre : tipo valor-por-defecto

Dirección. La dirección se representa mediante una palabra clave que precede al nombre de la operación. Si la palabra clave está ausente, entonces la dirección es **in**. Las opciones son **in**, **out**, **inout** y **return**. Los parámetros **return** suelen mostrarse en otra posición distinta en la signatura de las operaciones, lugar en que no es necesario marcar su dirección.

Nombre. El nombre se representa mediante una cadena.

Tipo. El tipo se denota como una cadena que es el nombre de una clase, una interfaz o un tipo de datos.

Valor por defecto. El valor se representa mediante una cadena de expresión. El lenguaje de la expresión debería ser conocido por la herramienta (y debe ser especificable para la herramienta) pero no se muestra en formato canónico.

Alcance. Si el alcance es de clase, entonces se subraya la cadena de operación. Si el alcance es de instancia, entonces no se subraya la cadena de la operación.

Dependencia paramétrica. Una dependencia paramétrica se representa mediante una flecha discontinua que va desde la operación que tiene el parámetro o la clase que contiene a esa operación hasta la clase del parámetro; la flecha tiene asociado el estereotipo «**parameter**».

Ejemplo

Matriz::transformación (in distancia: Vector, in ángulo: Real = 0): **return** Matriz

Se pueden omitir todas las etiquetas de dirección.

parámetro real

Véase argumento.

participantes

Denota la conexión de un elemento de un modelo con una relación o con una relación materializada. Por ejemplo, una clase participa en una asociación, un rol de clasificador participa en una colaboración.

patrón

Término que denota una colaboración parametrizada que representa un conjunto de clasificadores, relaciones y comportamientos parametrizados, que se pueden aplicar a múltiples situaciones enlazando elementos del modelo (clases normalmente) con los roles del patrón. Es una plantilla de colaboración.

Semántica

Los patrones representan una colaboración parametrizada que se puede emplear en múltiples ocasiones dentro de uno o más sistemas. Para ser un patrón, la colaboración tiene que ser utilizable en una amplia gama de situaciones, para justificar darle un nombre. Un patrón es una solución que demostradamente funciona en un cierto número de situaciones. No es necesariamente la única solución del problema, pero es una solución que ha sido efectiva en el pasado. Casi todos los patrones tienen ventajas y desventajas, que dependen de distintos aspectos del sistema global. El creador de modelos debe considerar estas ventajas y desventajas antes de tomar la decisión de utilizar un patrón.

Una colaboración parametrizada de UML representa las vistas estructural y de comportamiento de ciertas clases de patrones. Los patrones implican otros aspectos que no son modelados directamente por UML, tales como la lista de ventajas y desventajas de su utilización anterior. Hay muchos de estos aspectos que se pueden expresar en palabras. Véase [Gamma-95] para un tratamiento más completo de los patrones, así como un catálogo de patrones de diseño probados.

Generar colaboraciones a partir de patrones. Se puede utilizar una colaboración para especificar la implementación de estructuras de diseño. Es posible emplear la misma clase de colaboración en muchas ocasiones, parametrizando sus componentes. Un patrón es una colaboración parametrizada. En general, las clases de los roles que intervienen en la colaboración son parámetros. El patrón se instancia como una colaboración enlazando valores, normalmente clases, con sus parámetros. En el caso común de los roles parametrizados, la plantilla se enlaza especificando una clase para cada rol. Típicamente, los roles de asociación de un patrón no están parametrizados. Cuando se enlaza la plantilla, representan asociaciones implícitas entre las clases enlazadas con la colaboración, esto es, el enlazado de la plantilla para hacer una colaboración genera asociaciones adicionales.

Notación

El enlazado de un patrón para producir una colaboración se representa mediante una elipse discontinua que contiene el nombre del patrón (Figura 13.154). Se traza una línea discontinua desde el símbolo de enlazado del patrón hasta las clases (u otros elementos del modelo) que participen en la colaboración. Todas las líneas se etiquetan con el nombre del parámetro. En la mayoría de los casos el nombre de un rol de la colaboración se puede emplear como nombre de parámetro. Por tanto, un símbolo de enlazado de patrón puede mostrar la utilización de un patrón de diseño, junto con las clases que realmente aparecen en esa utilización del patrón. El

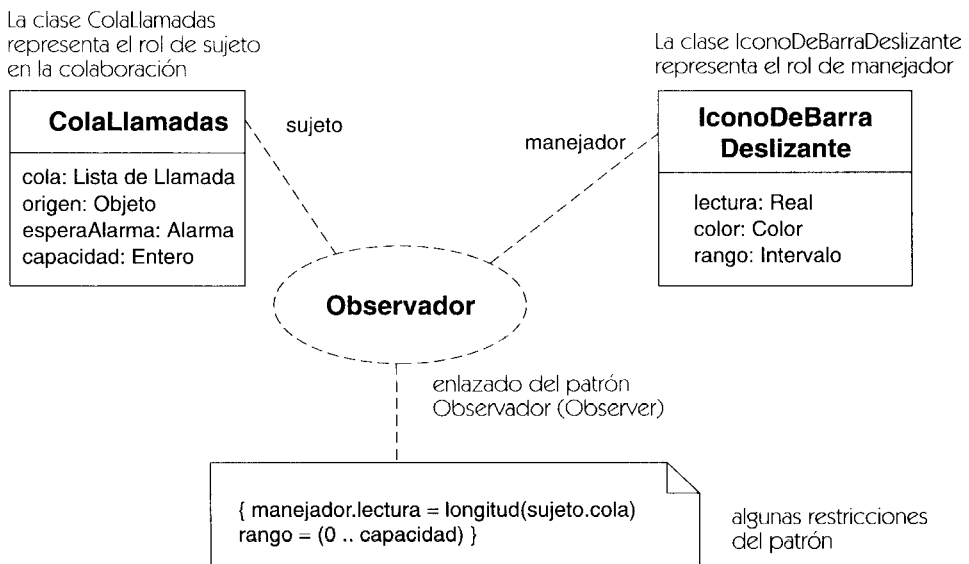


Figura 13.154 Enlazado de un patrón para formar una colaboración

enlazado del patrón no suele mostrar la estructura interna de la colaboración que se genera mediante el enlazado. Esto está implícito en el símbolo de enlazado.

permiso

Denota una clase de dependencia que concede al cliente permiso para utilizar el contenido del elemento proveedor (sometido a las declaraciones de visibilidad de los elementos del mismo).

Semántica

Los estereotipos de dependencia de permiso son `access`, `friend` e `import`. Nunca se utiliza un principio de dependencia al desnudo sin un estereotipo. Las dependencias de acceso y de importación se emplean en los paquetes. La dependencia de amistad se emplea con clases u operaciones como clientes y con clases como proveedores.

Notación

Las dependencias de permiso se representan mediante una flecha discontinua que va del cliente (el elemento al que se da permiso) al proveedor (el elemento que concede el permiso) con la correspondiente palabra reservada de estereotipo asociada a la flecha.

Elementos estándar

`friend`, `import`.

plantilla

Elemento parametrizado de un modelo. Para utilizarla, los parámetros tienen que estar asociados (en el tiempo de modelado) con valores reales. Sinónimo: elemento parametrizado.

Véase también enlace, elemento ligado.

Semántica

Una plantilla es el descriptor de un elemento con uno o más parámetros formales. Por tanto, define una familia de elementos potenciales, estando especificado cada uno de los elementos por el enlazado de los parámetros con valores reales. Típicamente, los parámetros son clasificadores que representan tipos de atributos, pero también pueden representar enteros o incluso operaciones. Los elementos subordinados dentro de la plantilla se definen en términos de los parámetros formales, así que también quedan enlazados cuando se enlaza la plantilla en sí como los valores reales.

Una plantilla de clase es el descriptor de una clase parametrizada. El cuerpo de una plantilla puede contener muchas apariciones de los parámetros formales, así como un elemento por defecto que representa la plantilla en sí. Las clases reales se producen enlazando los parámetros con valores. Los atributos y operaciones existentes dentro de la clase plantilla se pueden definir en términos de los parámetros formales. La clase plantilla también puede tener relaciones, tales como asociaciones y generalización, entre ella y uno de sus parámetros. Cuando se enlaza la plantilla, el resultado es una relación entre la clase de plantilla enlazada y las clases enlazadas con los parámetros de la relación.

Las clases de plantilla no son clases utilizables directamente porque poseen parámetros que no están enlazados. Es preciso enlazar los parámetros con valores reales para crear una clase real. Sólo una clase real podrá ser padre o destino de una asociación (una asociación monodireccional procedente de la plantilla y que llega a otra clase es admisible, sin embargo). Una clase de plantilla puede ser una subclase de una clase ordinaria, lo cual implica que todas las clases formadas al enlazar la plantilla son subclases de la clase dada. También puede ser hija de uno de los parámetros de plantilla; esto implica que la clase de plantilla enlazada es hija de la clase pasada como argumento.

Es posible aplicar la parametrización a otros elementos del modelo, tales como las colaboraciones e incluso paquetes completos. La descripción que se da aquí para las clases se aplica en la forma evidente a los otros elementos de modelado.

El contenido de una plantilla no está sometido directamente a las reglas de buena formación que afectan a los modelos. Esto se debe a que incluyen parámetros que no tienen una semántica completa mientras no hayan sido enlazados. Una plantilla es una especie de elemento de modelo de segundo nivel, no un elemento que modele directamente los sistemas sino uno que modela otros elementos del modelo. Por tanto, el contenido de la plantilla está fuera de la semántica del sistema. Los resultados de enlazar una plantilla son elementos ordinarios del modelo, sometidos a las reglas de buena formación y son elementos normales del sistema blanco. Se podrían derivar ciertas reglas de buena formación para plantillas a partir de las consideraciones de que los resultados enlazados tienen que ser bien formados, pero no vamos a intentar enumerarlas. En cierto sentido, cuando se enlaza una plantilla su contenido se duplica y los parámetros son sustituidos por los argumentos. El resultado pasa a formar parte del modelo efectivo como si se hubiese incluido directamente.

Hay otras clases de clasificadores, como los casos de uso y las señales, que también se pueden parametrizar. Las colaboraciones también se pueden parametrizar; entonces son patrones.

Notación

Se superpone un pequeño rectángulo discontinuo en la esquina superior derecha del rectángulo de clase u otro elemento de modelado. El rectángulo discontinuo contiene una lista de parámetros formales para la clase. Cada parámetro tiene su nombre y un clasificador. La lista no puede estar vacía (si lo estuviera, no sería una plantilla) aunque se puede suprimir en la representación. El nombre, los atributos y las operaciones de la clase parametrizada aparecen normalmente en el rectángulo de clase, pero también se pueden incluir apariciones de los parámetros formales. Las otras clases de elementos parametrizados se tratan análogamente. Las apariciones de los parámetros formales pueden producirse dentro del cuerpo de la

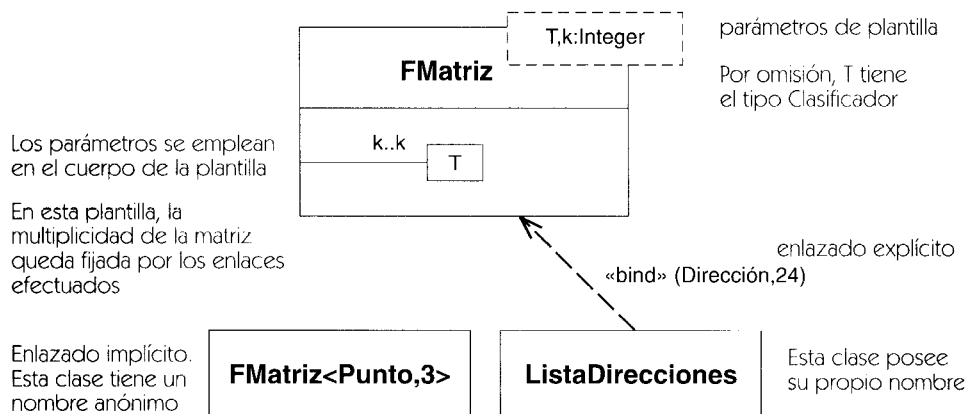


Figura 13.155 Notación de plantilla empleando un parámetro como referencia

plantilla, por ejemplo, para mostrar una clase relacionada que es identificada por uno de los parámetros.

Los parámetros tienen la sintaxis:

nombre :tipo

donde nombre es un identificador del parámetro, con alcance de plantilla; y

donde tipo es una cadena que denota una expresión de tipo para el parámetro.

Si se omite el nombre del tipo, se supone que se trata de una expresión de tipo que al resolverse producirá un clasificador, tal como un nombre de clase o un tipo de datos. Los demás tipos de parámetros (tales como **Entero**) deben mostrarse explícitamente y tienen que producir expresiones de tipo válidas cuando se resuelven.

La Figura 13.155 muestra una plantilla con un parámetro entero y un parámetro de clase. La plantilla tienen una asociación con uno de sus parámetros.

Discusión

El *modelo efectivo* es el modelo implícito que resulta de enlazar todas las plantillas; se trata del modelo implícito que describe el sistema. Los parámetros de plantilla no tienen significado dentro del modelo efectivo en sí, porque ya se habrán enlazado. Sólo se pueden utilizar dentro del alcance del cuerpo de la plantilla en sí. Esto basta para manejar los elementos constitutivos contenidos en el elemento parametrizado, por ejemplo, para atributos o operaciones contenidos en una clase parametrizada.

Los elementos que normalmente son externos respecto al elemento parametrizado plantean más dificultades. Por ejemplo, una clase puede tener asociación o generalizaciones que lleguen a otras clases. Si esas clases son parámetros de la plantilla, no puede formar parte del modelo efectivo, y sin embargo tampoco forman parte de una clase ordinaria. Por tanto, un elemento parametrizado contiene un *cuerpo* que representa un fragmento del modelo. El fragmento del modelo no forma parte de la plantilla en sí y puede incluir parámetros de plantilla tales como un

parámetro que represente una clase (que todavía no está especificada). Cuando se enlaza la plantilla, el cuerpo se copia implícitamente; los parámetros son reemplazados por los argumentos y la copia pasa a formar parte del modelo efectivo. Cada instanciación de la plantilla produce una adición al modelo efectivo. La Figura 13.156 muestra un ejemplo.

El cuerpo de la plantilla contiene implícitamente un elemento que representa el elemento de plantilla instanciado en sí; por ejemplo, la clase producida al enlazar una plantilla. Este elemento implícito se puede emplear para relaciones de construcción tales como asociaciones y generalizaciones con parámetros de plantilla. En la notación, los parámetros se dibujan dentro de los límites de la plantilla y hay una conexión al interior de los límites de la plantilla que denota una relación con el elemento instanciado implícito de la plantilla. Cuando se instancia la plantilla, éstas se transforman en relaciones del modelo efectivo entre el elemento de plantilla enlazado (recién instanciado) y los elementos (previamente existentes) que son argumentos de la plantilla.

Una plantilla puede ser hija de otro elemento. Esto significa que todo elemento enlazado generado a partir de ella es hijo del elemento dado. Por ejemplo, en la Figura 13.156, toda variable del tipo matricial (**VMatriz**) es una matriz (**Matriz**). Por tanto, **VMatriz<Punto>** es hijo de **Matriz**, **VMatriz<Dirección>** es hijo de **Matriz** y así sucesivamente.

Normalmente una plantilla no puede ser padre de otro elemento. Esto significaría que todo elemento generado al enlazar el padre sería padre del otro elemento. Aunque quizá alguien pudiera atribuir un significado a semejante situación, es poco probable.

Dos plantillas no pueden tener asociaciones entre sí, simplemente porque comparten el mismo nombre de parámetro. (Intentar hacer esto significaría que toda instanciación de la primera plantilla estaría relacionada con toda instanciación de la segunda plantilla, que no es lo que suele desearse normalmente. Esta cuestión ha sido malinterpretada por algunos autores en el pasado.) Los parámetros sólo tienen alcance dentro de su propia plantilla. Utilizar el mismo nombre de parámetro en dos plantillas no hace que sea el mismo parámetro. En general, si dos plantillas tienen elementos parametrizados que deben estar relacionados, entonces una de las plantillas debe instanciarse dentro del cuerpo de la otra. (Recuerde que una plantilla es instan-

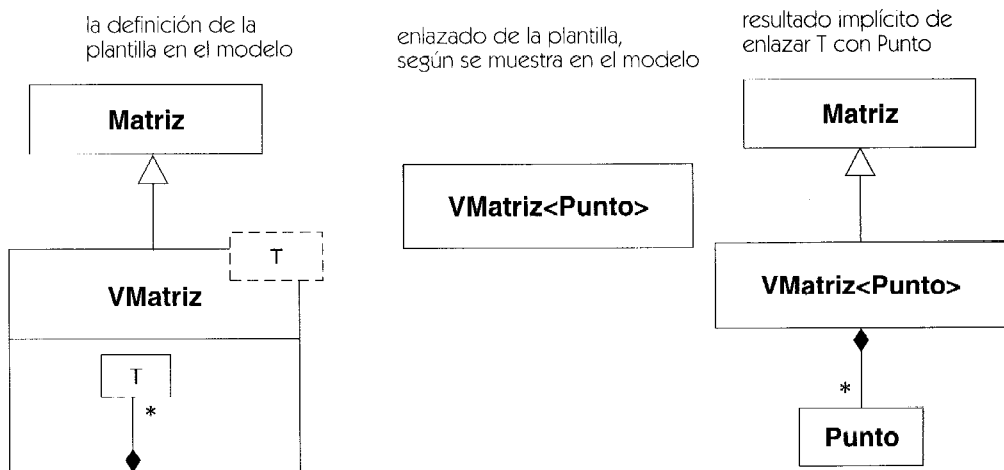
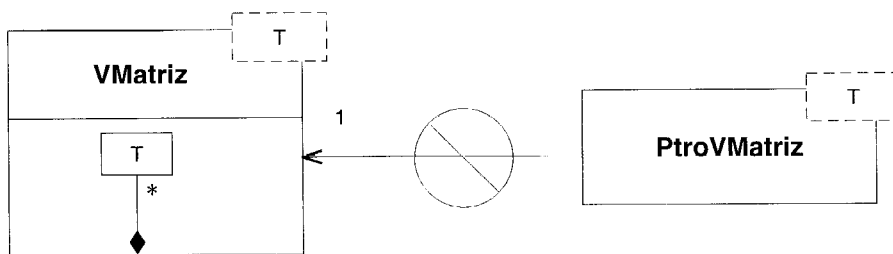


Figura 13.156 Plantilla con relación con uno de sus parámetros

¡MAL! Esto no tiene sentido. Las T no están relacionadas porque se encuentran en alcances distintos.



¡BIEN! Es preciso instanciar una plantilla para construir una asociación.

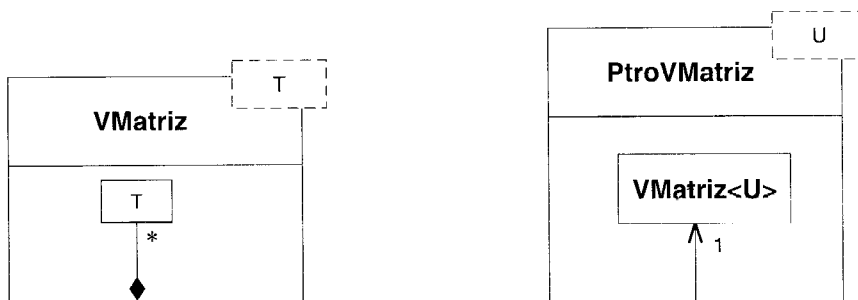


Figura 13.157 Asociaciones entre plantillas

ciada implícitamente dentro de su propio cuerpo. Por tanto, ambas plantillas son instanciadas efectivamente dentro del cuerpo y consiguientemente las relaciones son entre los elementos instanciados.) La Figura 13.157 muestra un intento incorrecto y otro correcto de definir semejante relación; en este caso, se ha empleado una clase parametrizada «**puntero**» que apunta a una matriz parametrizada de igual tipo.

Se puede utilizar un enfoque similar para declarar una clase parametrizada que sea hija de otra clase de plantilla enlazada con el mismo parámetro. El parámetro se utiliza para enlazar una copia de cada una de las otras plantillas. Entonces puede construirse una asociación entre las copias instanciadas de las plantillas. En la mayoría de los casos prácticos, no es necesario hacer esto porque se puede declarar la relación en el cuerpo de alguna de las plantillas.

polimórfico/a⁵

Indica una operación cuya implementación (un método o una máquina de estados disparada por un evento de llamada) puede ser proporcionada por una clase descendiente. Una operación que no sea polimórfica es una operación hoja.

Véase también operación abstracta, generalización, herencia, método.

⁵ Utilizaremos indistintamente los términos polimorfo y polimórfico para traducir el término inglés “polymorphic”. *N. del Revisor.*

Semántica

Si una operación es polimórfica, entonces se le puede proporcionar un método en una clase descendiente (tanto si ya se ha suministrado un método en la clase original como si no). En caso contrario, es preciso que esté disponible un método para esa operación en la clase en que se declara la operación y el método no podrá ser anulado en una clase descendiente. Se dice que un método está disponible cuando lo declara una clase o cuando es heredado de un antecesor. Una operación abstracta tiene que ser polimórfica (porque no tiene implementación directa). Una operación no es polimórfica si se declara como operación hoja.

Si una operación se declara polimorfa en una clase, esto es, si no se declara como hoja, entonces puede ser declarada como hoja en alguna clase descendiente. Esto evita que sea redefinida o anulada en algún descendiente posterior. Las operaciones hoja no se pueden declarar polimórficas en clases descendientes. No se podrán anular ni redefinir independientemente de la profundidad.

UML no impone unas reglas de combinación de métodos si se declara un método en una clase y se redefine este método en una clase descendiente (*véase* la discusión siguiente). Los mecanismos, como pueda ser la declaración de métodos antes, después y lateralmente, se pueden manejar tal como convenga para un lenguaje específico empleando valores etiquetados. Las acciones tales como llamar explícitamente al método heredado son, por supuesto, dependientes del lenguaje de acción, en todo caso.

Notación

Se declara una operación no polimorfa empleando la palabra reservada **{leaf}** (*hoja*). En caso contrario, se supone que es polimorfa.

Discusión

Toda operación abstracta es necesariamente polimórfica. En caso contrario, no se podría implementar en modo alguno. Bertrand Meyer las llama operaciones diferidas porque su especificación se define en una clase pero su implementación se retrasa hasta las subclases. Se trata de una utilización esencial de la herencia, quizá la más esencial de todas tanto para el modelado como para la programación. Mediante el uso de la herencia, se pueden aplicar parámetros a conjuntos de objetos de clases mixtas. El emisor no necesita conocer o determinar la clase a que pertenece cada objeto. Sólo es necesario que todos los objetos se adapten a una clase antecesora que define las operaciones deseadas. La clase antecesora no necesita definir las operaciones. Basta con que se limite a definir sus firmas. El emisor ni siquiera necesita conocer la lista de posibles subclases. Esto significa que se pueden añadir posteriormente nuevas subclases, sin perturbar las operaciones polimórficas que les son aplicables. El código fuente que invoca a las operaciones no necesita modificaciones cuando se añaden nuevas subclases. La posibilidad de añadir nuevas clases después de haber escrito el código original es uno de los pilares fundamentales de la tecnología orientada a objetos.

Una aplicación más problemática del polimorfismo es la sustitución (también la anulación puede dar problemas) de un método definido en una clase por un método distinto definido en una subclase. Suele decirse que esto es una forma de compartir, pero es peligroso. La redefinición no es incremental, así que todo lo que haya en el método original tiene que reproducirse en el método hijo, aunque sólo sea para efectuar un pequeño cambio. Esta clase de repetición es

proclive a errores. En particular, si el método original se cambia posteriormente, no hay garantía de que el método hijo sea modificado también. Hay ocasiones en que una subclase puede hacer uso de una implementación completamente distinta de cierta operación, pero hay muchos expertos que desaconsejan esta redefinición por el peligro que implica. En general, los métodos deberían ser bien completamente heredados o bien diferidos; en este último caso no hay implementación en la superclase así que no hay peligro de redundancia ni de inconsistencia.

Para permitir que una subclase extienda la implementación de una operación sin perder el método heredado, la mayoría de los lenguajes de programación ofrece alguna forma de combinación de métodos que hace uso del método heredado pero permite añadir código adicional. En C++ un método heredado puede ser invocado explícitamente por el nombre de la clase y de la operación, lo cual inserta rígidamente la jerarquía de clases en el código: no es un enfoque muy orientado a objetos. En SmallTalk, un método puede invocar a una operación de **super**, lo cual da lugar a que la operación sea manejada por el método heredado. Si cambia la jerarquía de clases, entonces la herencia sigue funcionando, posiblemente con un método de otra clase diferente. Sin embargo, el método que anula debe proporcionar explícitamente una llamada a **super**. Los errores se producen, porque los programadores se olvidan de insertar las llamadas cuando se produce un cambio. Por último, CLOS ofrece unas reglas muy generales y complicadas para la combinación automática de métodos que puede invocar a varios métodos durante la ejecución de una sola operación de llamada. La operación global está implementada a partir de varios fragmentos en lugar de estar limitada a un único método. Esto es muy general pero resulta más difícil de manejar para el usuario.

UML no impone un único enfoque de combinación de métodos. La combinación de métodos es una cuestión de variación semántica. Se puede emplear cualquiera de estos enfoques. Si el lenguaje de programación no es muy potente en lo tocante a la combinación de métodos, entonces es posible que una herramienta de modelado pueda servir de ayuda para generar el código adecuado en ese lenguaje de programación, o quizá pueda emitir advertencias relativas a posibles errores causados por inacción si se emplea la anulación de métodos.

postcondición

Es una restricción que tiene que satisfacerse cuando se produzca la terminación de una operación.

Semántica

Una postcondición es una expresión booleana que tiene que ser verdadera una vez finalizada la ejecución de una operación. Se trata de una aserción⁶, no es una sentencia ejecutable. Dependiendo de la forma exacta de la expresión, quizá sea posible verificarla automáticamente por anticipado. Puede ser útil verificar la postcondición después de la operación, pero esto entra dentro de los límites de la depuración de programas. Se supone que la condición es verdadera y cualquier otra cosa es un error de programación. Una postcondición es una restricción que afecta a quien implementa una operación. Si no se satisface, entonces la operación se ha implementado de forma incorrecta.

Véase también invariante, precondition.

⁶ El término "assertion" también puede ser traducido como "aserto" o "aseveración", aunque nosotros hemos preferido el más común de "aserción". *N. del Revisor.*

Estructura

Las postcondiciones se modelan como las restricciones, con el estereotipo «**postcondition**», que está asociado a una operación.

Notación

Se puede mostrar una postcondición en una nota con la palabra reservada «**postcondition**». La nota está asociada a la operación afectada.

Ejemplo

La Figura 13.158 muestra una postcondición aplicada a una operación de clasificación de una matriz. El nuevo valor de la matriz (a') está relacionado con el valor original (a). Este ejemplo se expresa en lenguaje natural estructurado. También se podría hacer la especificación en un lenguaje más formal.

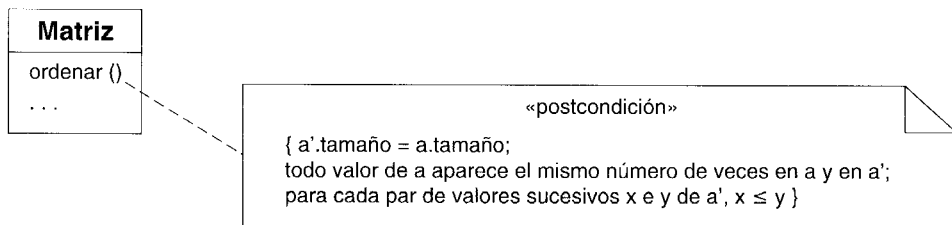


Figura 13.158 Postcondición

precondición

Una restricción que tiene que ser cierta cuando se invoca a una operación.

Semántica

Se trata de una expresión booleana que tiene que ser verdadera cuando se llama a una operación. Es responsabilidad del emisor satisfacer esa condición. No se trata de una condición que tenga que verificar el receptor. Una precondición no es una condición de guarda; es una condición que tiene que *ser* cierta y no una forma de ejecutar opcionalmente una operación. Puede resultar útil verificar la precondición al principio de la operación para más fiabilidad, pero esto entra en los límites de la depuración de programas. Se supone que la condición es verdadera y cualquier otra circunstancia es un error de programación. Si no se cumple la condición, no se puede hacer afirmación alguna con respecto a la integridad de la operación o del sistema. Tiene probabilidades de ser un fracaso absoluto. En la práctica, al verificar explícitamente las precondiciones en el receptor se pueden detectar muchos errores.

Véase también invariante, postcondición.

Estructura

Las precondiciones se modelan como restricciones con el estereotipo «**precondition**» que se asocia a una operación.

Notación

Se puede mostrar una precondición en una nota con la palabra reservada «**precondition**». La nota se asocia a la operación afectada.

Ejemplo

La Figura 13.159 muestra una precondición aplicada a un operador de producto matricial.

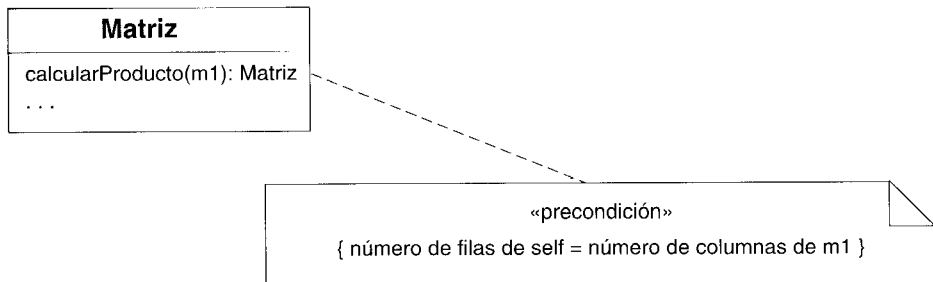


Figura 13.159 Precondición

principio de capacidad de sustitución

Principio consistente en que dada la declaración de una variable o parámetro cuyo tipo declarado es X, toda instancia de elemento que sea descendiente de X puede utilizarse en lugar del valor real sin violar la semántica de la declaración ni de su utilización. En otras palabras, se puede reemplazar una instancia de un elemento antecesor por una instancia de un elemento descendiente. (Atribuido a Barbara Liskov.)

Véase también generalización, herencia de implementación, herencia, herencia de interfaz, polimórfico/a, herencia privada.

Discusión

El propósito es asegurar que las operaciones polimórficas funcionen libremente. No se trata de un principio lógico sino más bien de una regla práctica de programación que proporciona un cierto grado de encapsulamiento. La relación de generalización apoya al principio de capacidad de sustitución.

La consecuencia del principio de capacidad de sustitución es que un hijo no puede eliminar ni redefinir propiedades de su padre. En caso contrario, no se podría utilizar al hijo como sustituto en una situación en que se hubiera declarado la utilización del padre.

privado

Se trata de un valor de visibilidad que indica que el elemento dado no es visible fuera de su propio espacio de nombres, ni siquiera para los descendientes de ese espacio de nombres.

proceso de desarrollo

Un conjunto de pautas y un conjunto de actividades de trabajo parcialmente ordenadas pensadas para producir software de una manera controlada, reproducible. El propósito de un proceso de desarrollo de software es asegurar el éxito y la calidad de un sistema acabado.

Véase también etapas de modelado.

Discusión

UML es un lenguaje de modelado, no un proceso, y su propósito es describir los modelos que se pueden producir por varios procesos de desarrollo. Para la estandarización, es más importante describir los artefactos resultantes de un desarrollo que el proceso de producirlos. Eso es porque hay muchas buenas formas de construir un modelo, y un modelo acabado puede ser utilizado sin saber cómo fue producido. Sin embargo, UML está pensado para apoyar una amplia gama de procesos.

Para más detalles del proceso iterativo, incremental, guiado por casos de uso, centrado en la arquitectura que promueven los autores de este libro. *Véase* [Jacobson-99].

La relación de las etapas de modelado y las fases de desarrollo

Las etapas de modelado encajan dentro de un proceso de desarrollo iterativo, que tiene las fases de inicio, elaboración, construcción, y transición (fase). Las fases son secuenciales dentro de una entrega de una aplicación, pero cada fase incluye una o más iteraciones. Dentro de una iteración, los elementos individuales del modelo se mueven a lo largo del camino desde el análisis hacia el despliegue, cada uno a su propio ritmo. Aunque las fases del desarrollo y las etapas de modelado no se sincronizan, hay una correlación.

En las fases anteriores del desarrollo y las iteraciones anteriores de una fase, hay más énfasis en las etapas de modelado anteriores.

La Figura 13.160 muestra el equilibrio del esfuerzo durante las fases e iteraciones sucesivas. Durante la inicio, el foco de interés está principalmente en análisis, con un esqueleto de los elementos que progresa hacia el diseño y la implementación durante la elaboración. Durante la construcción y la transición, todos los elementos deben, eventualmente, llegar a su terminación.

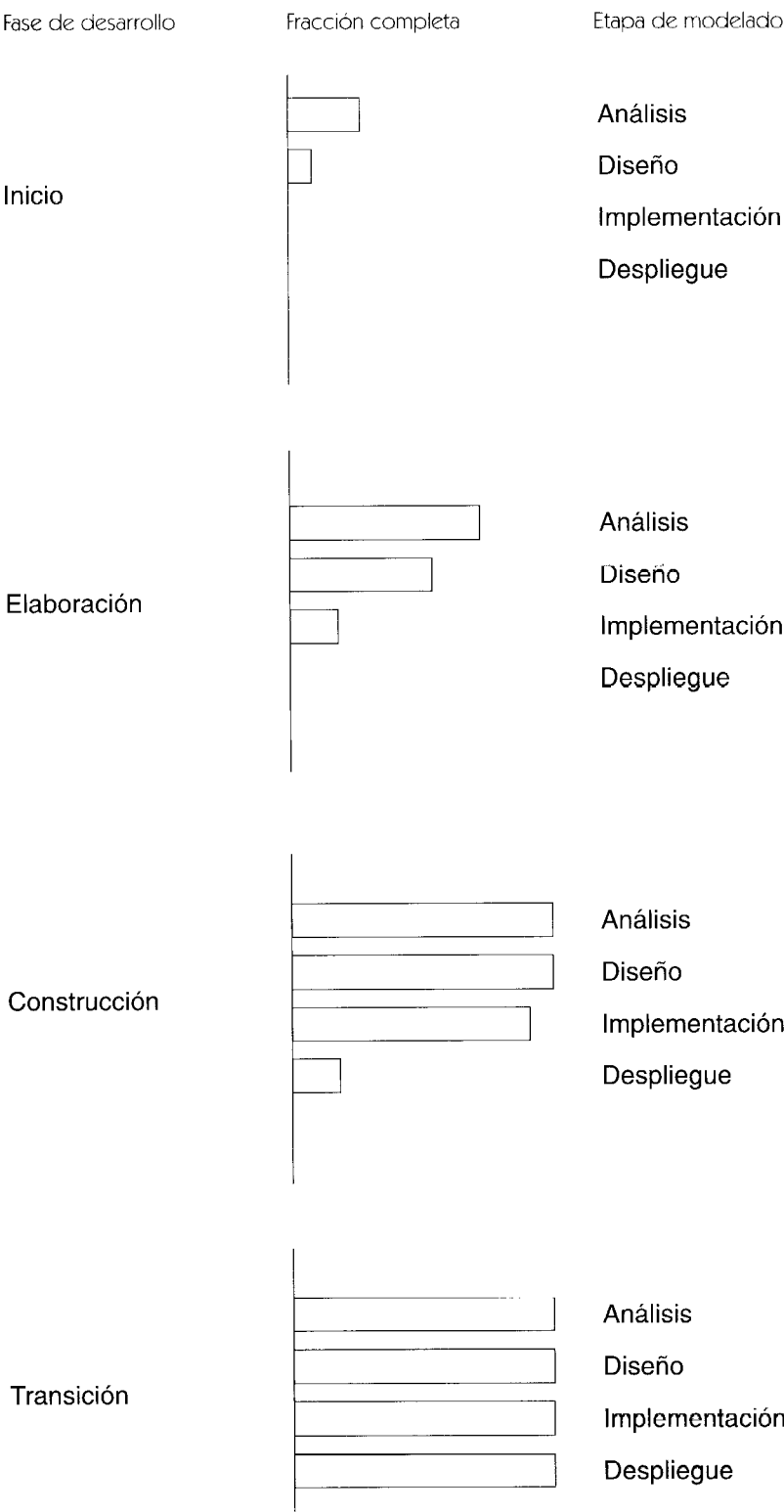


Figura 13.160 Progreso después de cada fase de desarrollo

proceso/procesar

1. Unidad compleja de concurrencia y ejecución en un sistema operativo. Véase hilo, que incluye los procesos complejos y ligeros. Si es necesario, se puede hacer una distinción de implementación empleando estereotipos.
2. Un proceso de desarrollo de software; son los pasos y directrices mediante los cuales se desarrolla un sistema.
3. Ejecutar un algoritmo o en general manejar algo dinámicamente.

producto

Dícese de los resultados del desarrollo, tales como modelos, código, documentación y planes de trabajo; también puede denotar un producto de trabajo.

propiedades

Término genérico que denota un valor con nombre que aporta información acerca de un elemento del modelo. Las propiedades tienen impacto semántico. Ciertas propiedades están definidas en UML, otras pueden ser definidas por el usuario.

Véase atributo, relación, valor etiquetado.

Semántica

Entre las propiedades se cuentan los atributos incorporados (definidos por UML) así como los valores etiquetados (definidos por el usuario) y las relaciones (definidas por el usuario) asociadas a un elemento. Desde el punto de vista de un usuario, suele carecer de importancia el que una propiedad sea incorporada o esté implementada como un valor etiquetado.

Discusión

Obsérvese que se utiliza la palabra *propiedad* en un sentido general, para denotar cualquier valor que esté asociado a un elemento del modelo, incluyendo los atributos, asociación y valores etiquetados. En este sentido, puede incluir los valores alcanzables indirectamente que se pueden hallar comenzando en un determinado elemento.

protegida

Dícese de aquel valor de visibilidad que indica que el elemento dado sólo es visible fuera de su propio espacio de nombres para los descendientes del espacio de nombres.

proveedor

Elemento que proporciona servicios que pueden ser invocados por otros. Contrastar con: cliente. En la notación, el proveedor aparece en la punta de flecha de una flecha discontinua de dependencia.

Véase dependencia.

proyección

Correspondencia entre un conjunto y un subconjunto del mismo. La mayoría de los modelos y de los diagramas son proyecciones del conjunto completo de información que está potencialmente disponible.

pseudoatributo

Denota un valor relacionado con una clase que se comporta como un atributo, a saber, que tiene un valor exclusivo para cada instancia.

Véase también discriminador, nombre de rol.

Semántica

Entre los pseudoatributos se cuentan los nombres de roles de asociación y los discriminadores de generalización. Un nombre de rol de asociación es un pseudoatributo de la clase que se encuentra al otro extremo de la asociación. Un discriminador de generalización es un pseudoatributo del elemento padre. En todos los elementos hijos, el valor del discriminador es el nombre del elemento hijo.

Se puede utilizar un pseudoatributo como nombre en una expresión para recuperar un valor de un objeto. Dado que los nombres de atributos y los nombres de pseudoatributos se pueden emplear en las expresiones, se encuentran en el mismo espacio de nombres y tienen que ser únicos en ese espacio de nombres. Los nombres también tienen que ser únicos respecto a los nombres de atributos y pseudoatributos heredados.

pseudoestado

Denota un vértice de una máquina de estados que posee la forma de un estado pero no se comporta como un estado completo. Véase estado abreviado externo, estado de historia, estado de unión o conjunción, estado inicial.

Semántica

Cuando un pseudoestado está activado, hay una máquina de estados que no ha finalizado su paso que se ejecuta hasta finalizar y que no procesa eventos. Entre los pseudoestados se cuentan el es-

tado inicial, el estado de unión o conjunción, el estado abreviado externo y el estado de historia. Los pseudoestados se utilizan para encadenar segmentos y una transición a uno de ellos implica otra transición automática adicional a otro estado sin necesidad de que intervenga un evento.

Los estados finales y de sincronización no son pseudoestados. Son estados especiales que pueden permanecer activos cuando una máquina de estados ya ha finalizado su paso que se ejecuta hasta finalizar, pero tienen restricciones relativas a las transiciones que pueden partir de ellos.

pública

Dícese de aquel valor de visibilidad que indica que el elemento dado es visible fuera de su propio espacio de nombres.

punto de extensión

Un marcador con nombre que se refiere a una localización o a un conjunto de localizaciones dentro de la secuencia de comportamiento para un caso de uso, en el cual se puede insertar comportamiento adicional. Una declaración de punto de extensión abre la posibilidad de extensión al caso de uso. Un segmento de inserción es una secuencia de comportamiento en un caso de uso de extensión (un caso de uso relacionado con un caso de uso base por una relación de extensión). La relación de extensión contiene una lista de nombres de puntos de extensión que indican el lugar donde los segmentos de la inserción del caso de uso extendido insertan su comportamiento.

Véase también extender, caso de uso.

Semántica

Un punto de extensión tiene un nombre y se refiere a un conjunto de una o más localizaciones dentro de una secuencia de comportamiento de un caso de uso. Un punto de extensión puede referirse a una sola localización entre dos pasos del comportamiento dentro de un caso de uso. Además, puede referirse a un conjunto de localizaciones discretas. Puede también referirse a una región dentro de una secuencia de comportamiento (éste no es nada más que el conjunto de todas las localizaciones entre los pasos de la secuencia).

Una localización es un estado dentro de la máquina de estados que describe un caso de uso, o el equivalente de una descripción diferente —entre dos sentencias de una lista de sentencias o entre dos mensajes en una interacción.

Una relación de extensión contiene una condición opcional y una lista de referencias a puntos de extensión iguales en número al número de los segmentos de inserción en el caso de uso extendido. Un segmento de inserción puede ser realizado si la condición está satisfecha mientras que una instancia del caso de uso está ejecutando el caso de uso base en cualquier localización en el punto de extensión que corresponde al segmento de la inserción.

La localización de un punto de extensión puede ser cambiada sin afectar su identidad. El uso de los puntos de extensión con nombre separa la especificación de las secuencias de los detalles internos del caso de uso base. El caso de uso base puede ser modificado o ser cambiado sin afectar las extensiones. Por otra parte, un punto de la extensión se puede mover dentro de la base sin afectar la relación o el caso de uso extendido. Como con todos los tipos de modularidad, esta independencia requiere una buena elección de los puntos de extensión y no está garantizada bajo todas las circunstancias.

Notación

Los puntos de extensión para un caso de uso se pueden enumerar en un compartimento llamado **puntos de extensión**.

Los puntos de extensión deben también referirse a localizaciones dentro de la secuencia de comportamiento del caso de uso. El beneficio neto es que los nombres del punto de extensión sirven como etiquetas para los estados, las declaraciones, y las regiones dentro de la secuencia de comportamiento. Esto no significa que deban estar presentes en el texto de la secuencia de comportamiento original.

Los puntos de extensión son nombres que permiten separar la localización de la opción para realizar una inserción. Una herramienta de edición puede poner un punto de extensión en un recubrimiento del texto de la secuencia de comportamiento sin modificar el texto original, o puede tener una tabla separada de los nombres de puntos de extensión distribuida en declaraciones (por número de declaración, etiquetas internas, o gráficos directos). En cualquier caso, el efecto neto corresponde a la localización de cada nombre de punto de extensión entre uno o más pasos en las secuencias de comportamiento. En el ejemplo de pseudocódigo en la Figura 13.161 un nombre de punto de extensión entre símbolos de < y > refiriéndose a una localización en la secuencia de comportamiento. Hay muchas sintaxis posibles para localizar puntos de extensión dentro de secuencias de comportamiento, dependiendo del lenguaje utilizado para especificar la secuencia. Esto es un ejemplo, pero no la única forma.

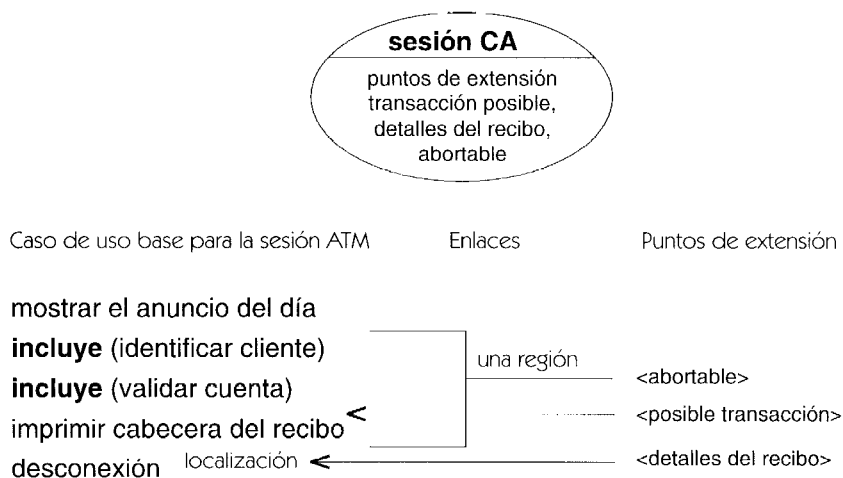


Figura 13.161 Declaración de puntos de extensión

punto de variación semántica

Punto de variación en la semántica de un metamodelo. Proporciona un grado de libertad deliberado para la interpretación de la semántica del metamodelo.

Discusión

Una misma semántica de ejecución no resulta adecuada para todas las posibles aplicaciones. Los distintos lenguajes de programación y los diferentes propósitos requieren variaciones de la semántica; algunas de ellas son sutiles y otras bastas. Un punto de variación semántica es un asunto en que distintos creadores de modelos o diferentes entornos de ejecución discrepan con respecto a la semántica específica, frecuentemente con buenas razones. Al identificar y dar nombre a los puntos de variación semántica, se pueden evitar las discusiones acerca de la semántica "correcta" de un sistema.

Por ejemplo, la opción de permitir o no la clasificación múltiple o la clasificación dinámica es un punto de variación semántica. Cada opción es una variación semántica.

Entre otros ejemplos de puntos de variación semántica se incluye si una llamada puede proporcionar más de un valor y si las clases existen durante el tiempo de ejecución como objetos reales.

realización

Relación entre una especificación y su implementación; indicación de la herencia de comportamiento sin la herencia de estructura.

Véase también interfaz.

Semántica

Las especificaciones describen el comportamiento o estructura de algo sin determinar cómo se implementará el comportamiento. Una implementación proporciona los detalles relativos a la forma de implementar el comportamiento de una forma computable efectivamente. La relación entre un elemento que especifica comportamiento y otro que proporciona una implementación se denomina realización. En general, hay muchas maneras de realizar una especificación. De forma similar, un elemento puede realizar más de una especificación. La realización es, por tanto, una relación muchos-a-muchos entre elementos.

El significado de la realización es que el elemento cliente tiene que admitir todo el comportamiento del elemento proveedor pero no tiene por qué satisfacer su estructura o su implementación. Un clasificador cliente, por ejemplo, tiene que admitir las operaciones del clasificador proveedor; además tiene que admitir todas las máquinas de estados que especifiquen el comportamiento externo del proveedor. Pero todos los atributos, asociaciones, métodos o máquinas de estados del proveedor que especifiquen la implementación son irrelevantes para el cliente. Observe que el cliente no hereda realmente las operaciones del proveedor. Tiene que declararlas él mismo o heredarlas de algún antecesor para que queden cubiertas todas las operaciones del pro-

veedor. En otras palabras, el proveedor de una realización indica qué operaciones tienen que estar presentes en el cliente, pero el cliente es quien tiene la responsabilidad de aportarlas.

Ciertas clases de elementos, tal como las interfaces y los casos de uso, están destinados a especificar el comportamiento y no contienen información de implementación. Otras clases de elementos, tales como las clases, están destinadas a implementar el comportamiento. Contienen información de implementación pero también se puede utilizar de forma más abstracta como especificadores. Normalmente, la realización relaciona un elemento de especificación, tal como un caso de uso o una interfaz, con un elemento de implementación, tal como una colaboración o una clase. Es posible utilizar un elemento de implementación, tal como una clase, para la especificación. Se puede situar en el lado de especificación de una relación de realización. En este caso, sólo las partes de especificación de la clase proveedora afectarán al cliente. Entonces, hablando con más precisión, la realización es una relación entre dos elementos en los cuales las partes de especificación de comportamiento externo de uno de ellos restringen la implementación del otro. Se podría pensar que se trata de una herencia de comportamiento sin herencia de estructura o de implementación (y con la necesidad de declarar realmente las operaciones por parte del cliente).

Si el elemento de especificación es una clase abstracta sin atributos, sin asociaciones y únicamente con operaciones abstractas, entonces toda especialización de la clase abstracta realiza la clase abstracta, porque nada hay que heredar salvo la especificación.

El elemento que hace la implementación tiene que admitir todo el comportamiento incluido en el elemento que hace la especificación. Por ejemplo, una clase tiene que contener todas las operaciones de aquellas interfaces que realiza, con una semántica que sea consistente con todas las especificaciones que requieran las interfaces. La clase puede implementar operaciones adicionales y la implementación de las operaciones puede hacer cosas adicionales, siempre y cuando no se violen las especificaciones de operación de las interfaces.

Notación

La relación de realización se muestra mediante una ruta discontinua con una punta de flecha triangular hueca adyacente al elemento que proporciona la especificación y con el origen en el elemento que proporciona la implementación (Figura 13.162).

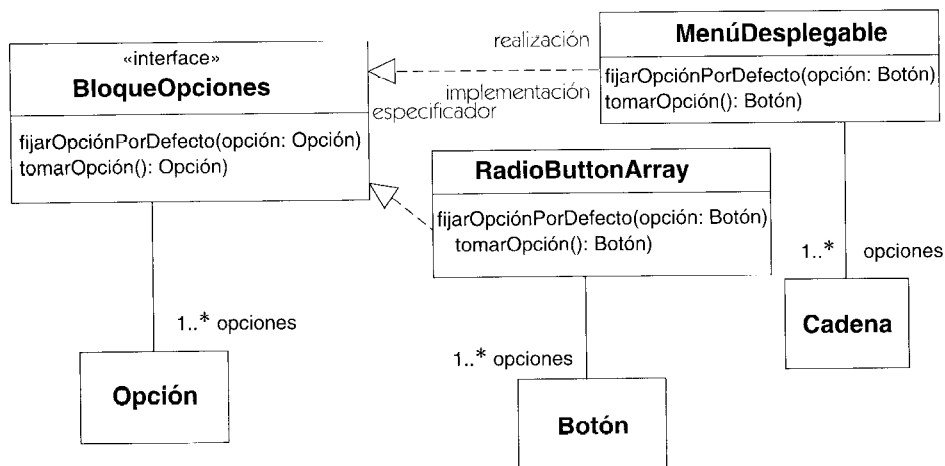


Figura 13.162 Relación de realización

Discusión

Otro caso importante es la realización de un caso de uso mediante una colaboración (Figura 13.163). Los casos de uso especifican secuencias de funcionalidad y comportamiento visibles externamente pero no proporcionan una implementación. Una colaboración describe los objetos que implementan el comportamiento del caso de uso y la forma en que interaccionan para hacerlo. Normalmente, una colaboración implementa un caso de uso, pero una colaboración se puede implementar empleando colaboraciones subordinadas, cada una de las cuales hace una parte del trabajo. Los objetos y las clases que se utilizan para implementar una colaboración suele aparecer también en otras colaboraciones.



Figura 13.163 Realización de un caso de uso por colaboración

Toda clase de la colaboración dedica una parte de su funcionalidad al caso de uso que se está implementando. Por tanto, el caso de uso será implementado eventualmente por partes en varias clases.

realizar

Proporcionar la implementación de un elemento de especificación.

Véase realización.

recepción

La declaración de que un clasificador está preparado para reaccionar cuando reciba una señal. Es un miembro de un clasificador.

Semántica

Una recepción es una declaración que indica que un clasificador está preparado para admitir una instancia de una señal y reaccionar frente a ella. Una recepción es algo parecido a una operación. Declara la signatura de un mensaje que admite el clasificador y especifica su significado.

Estructura

Una recepción posee las propiedades siguientes:

polimorfismo	Denota si la respuesta del clasificador frente a la señal es o no siempre la misma. Se codifica mediante la propiedad isPolymorphic , mediante los valores siguientes:
--------------	---

true	La respuesta es polimorfa: puede depender del estado y también puede ser anulada por un descendiente.
false	La respuesta debe ser la misma, independientemente del estado, y no puede ser anulada por un descendiente. El efecto neto es que tiene que haber una única transición de toda la máquina de estados que maneja el evento.
señal	Denota la señal a la cual responderá el clasificador.
especificación	Una expresión que enuncia los efectos que causa la recepción de la señal.

Notación

La recepción puede mostrarse en la lista de operaciones de una clase o interfaz empleando la sintaxis correspondiente a una operación con la palabra reservada «**signal**» delante del nombre de la señal.

Alternativamente, se puede situar una lista de firmas de señal en su propio compartimento; el compartimento tiene el nombre **Señales**. Se muestran las dos formas en la Figura 13.164.

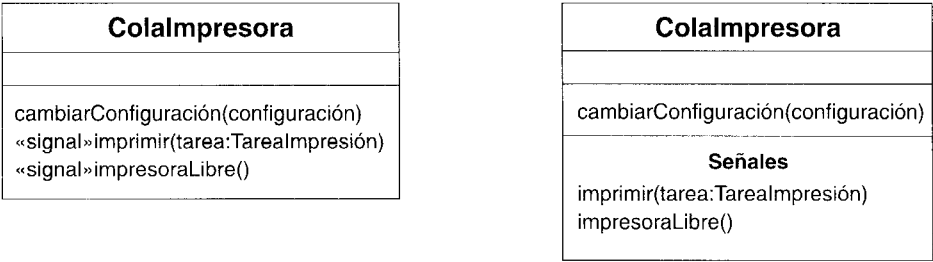


Figura 13.164 Hay dos maneras de indicar la recepción de una señal

receptor

Dícese de aquel objeto que maneja una instancia de mensaje que se le pasa desde un objeto emisor.

recibir

Manejar una instancia de mensaje que se nos pasa procedente de un objeto emisor.

Véase emisor, receptor.

referencia

Alude a una forma de denotar un elemento del modelo; suele llamarse puntero.

Semántica

Los elementos del modelo están conectados mediante dos metarrelaciones: propiedad y referencia. La propiedad es la relación que existe entre un elemento y sus partes constitutivas, las partes que están definidas en su interior y que él posee. La relación de propiedad forma un árbol estricto. Los elementos contenidos están subordinados al elemento contenedor. La propiedad, el control de configuración y el almacenamiento de modelos están basados en la jerarquía de contención.

La referencia es una relación entre elementos de un mismo nivel de detalle o bien entre diferentes contenedores. Por ejemplo, la referencia es la relación que existe entre una asociación y las clases participantes en ella, entre un atributo y la clase o tipo de dato que es una propiedad de tipo, entre una plantilla ligada y los valores de sus argumentos. Para que sea posible una referencia, el elemento del que se haga la referencia tiene que ser visible para el elemento que la hace. En general, esto significa que el paquete que contiene el origen de la referencia debe poder ver el paquete que contiene el destino de la referencia. Esto requiere una relación adecuada de acceso o de importación entre los paquetes. También requiere que el elemento al que se haga referencia tenga una configuración de visibilidad que le permita ser visto fuera de su paquete, a no ser que la referencia tenga lugar en el mismo paquete.

Obsérvese que la referencia es una relación interna de metamodelo, no una relación visible por el usuario; se emplea para construir las demás relaciones.

refinamiento

Denota una relación que representa una especificación más completa de algo que ya se ha especificado con un cierto nivel de detalle o en un nivel semántico diferente.

Véase también abstracción.

Semántica

El refinamiento es una conexión histórica o computable entre dos elementos que poseen una correspondencia (no necesariamente completa) entre sí. Con frecuencia, los dos elementos se encuentran en modelos diferentes. Por ejemplo, una clase de diseño puede ser un refinamiento de una clase de análisis; tiene los mismos atributos lógicos pero sus clases pueden provenir de una biblioteca de clases específica. Sin embargo, un elemento puede refinar a otro en un mismo modelo. Por ejemplo, una versión optimizada de una clase es un refinamiento de la versión sencilla pero ineficiente de esa misma clase. La relación de refinamiento puede contener una descripción de la correspondencia, que se puede escribir en un lenguaje formal (tal como OCL o algún lenguaje lógico o de programación). También puede ser un texto informal (lo cual, evidentemente, impide toda computación automática pero puede ser útil en las fases iniciales del desarrollo). Se puede utilizar el refinamiento para modelar la elaboración paso a paso del desarrollo, optimización, transición y el marco de trabajo.

Estructura

El refinamiento es una clase de dependencia. Relaciona a un cliente (el elemento que está más desarrollado) con un proveedor (el elemento que es la base del refinamiento).

Notación

Los refinamientos se indican mediante una flecha de dependencia (una flecha discontinua con la punta en el elemento más general y el origen en el elemento más específico) que posee la palabra reservada «**refine**». La correspondencia puede estar asociada a la ruta de dependencia mediante una línea discontinua conectada a una nota. Se han propuesto diferentes tipos de refinamientos; es posible indicarlos mediante más estereotipos. En muchos casos, el refinamiento conecta elementos de diferentes modelos y, por tanto, no será visible gráficamente. Con cierta frecuencia, sólo está implícito.

Ejemplo

La optimización es una clase típica de refinamiento. La Figura 13.165 muestra un tablero de ajedrez que posee una sencilla representación en el modelo de análisis, pero otra representación mucho más elaborada y oscura en el modelo de diseño. La clase de diseño no es una especialización de la clase de análisis, porque tiene una forma completamente diferente. La clase del modelo de análisis y la del modelo de diseño tienen el mismo nombre, porque representan un mismo concepto en dos niveles semánticos diferentes.

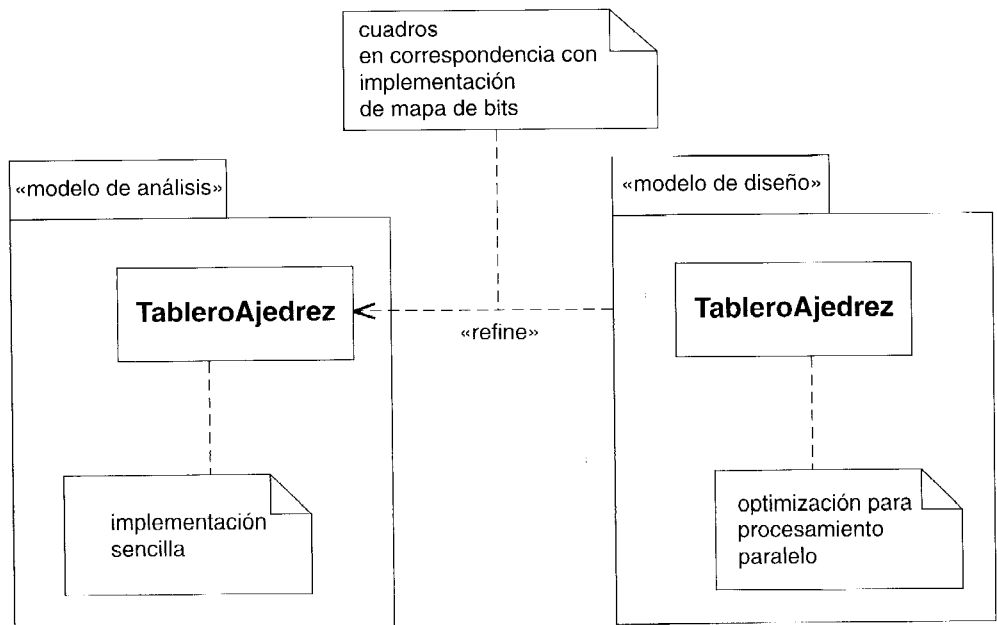


Figura 13.165 Refinamiento

refine

Palabra clave correspondiente a una dependencia de refinamiento en la notación.

relación

Dícese de una conexión semántica materializada entre elementos de un modelo. Entre las clases de relaciones se cuentan la asociación, generalización, metarelación, flujo y distintos grupos que están agrupados con el nombre de dependencia.

Semántica

La Tabla 13.2 muestra las distintas clases de relaciones que hay en UML. La primera columna (clase) denota los agrupamientos en que se organizan en el metamodelo. La segunda columna

Tabla 13.2 Relaciones en UML

<i>Clase</i>	<i>Variedad</i>	<i>Notación</i>	<i>➤ Símbolo o palabra reservada</i>
abstracción	derivación	dependencia	« derive »
	realización	realización	----- ▷
	refinamiento	dependencia	« refine »
	traza	dependencia	« trace »
asociación		asociación	————
enlazado		dependencia	« bind » (parametros _{list,})
extensión		dependencia	« extend » (extensión puntos _{lista,})
flujo	conversión	dependencia	número-secuencia: « become »
	copia	dependencia	número-secuencia: « copy »
generalización		generalización	——▷
inclusión		dependencia	« include »
metarelación	instancia	dependencia	« instanceOf »
	supratipo	dependencia	« powertype »
permiso	acceso	dependencia	« access »
	amigo	dependencia	« friend »
	importación	dependencia	« import »
utilización	llamada	dependencia	« call »
	instanciación	dependencia	« instantiate »
	parámetro	dependencia	« parameter »
	envío	dependencia	« send »

(variedad) muestra las distintas clases de relaciones. La tercera columna (notación) muestra la notación base para cada relación: la asociación es una ruta continua, la dependencia es una flecha discontinua y la generalización es una ruta continua con punta de flecha triangular. La cuarta columna (palabra reservada) muestra las palabras reservadas de texto y la sintaxis adicional de aquellas relaciones que la requieren.

repositorio

Lugar de almacenamiento para modelos, interfaces e implementaciones; forma parte de un entorno para manipular artefactos de desarrollo.

requisito

Una característica, propiedad o comportamiento que se desea para el sistema.

Semántica

Un requisito de texto se puede modelar como un comando al que se adjunta el estereotipo «**requirement**».

Discusión

El término *requisito* es una palabra del lenguaje natural que se corresponde con una gama de estructuras de UML destinadas a especificar las características deseadas para un sistema. Los requisitos correspondientes a transacciones visibles por parte del usuario suelen capturarse como casos de uso. Los requisitos no funcionales, como las métricas de rendimiento y de calidad, se pueden capturar en forma de enunciados de texto que eventualmente llegarán a los elementos del diseño final. Los comentarios y restricciones de UML se pueden emplear para representar requisitos no funcionales.

responsabilidad

Contrato u obligación de una clase u otro elemento.

Semántica

Una responsabilidad puede representarse como un estereotipo en un comentario. El comentario se asocia a la clase u otro elemento que tenga la responsabilidad. Ésta se expresa en la forma de una cadena de texto.

Notación

Las responsabilidades se pueden mostrar en un compartimento con nombre situado dentro del rectángulo que es el símbolo de un clasificador (Figura 13.166).

LíneaCrédito	
adeudar (): Booleano abonar () ajustar (límite: Dinero, código: Cadena)	compartimento de operación predefinido
responsabilidades adeudar cargos pagar débitos rechazar cargos por encima del límite ajustar límites con autorización notificar a contabilidad si moroso llevar la cuenta del límite de crédito llevar la cuenta de cargos actuales	compartimento de responsabilidad definido por el usuario lista de responsabilidades

Figura 13.166 Compartimento de responsabilidades

restricción

Una condición semántica o limitación representada como expresión. Ciertas restricciones se predefinen en UML, otras se pueden definir por los modeladores. Las restricciones son uno de tres mecanismos de extensión en UML.

Véase también expresión, estereotipo, valor etiquetado.

Véase Capítulo 14, Elementos Estándar, para una lista de restricciones predefinidas.

Semántica

Una restricción es una condición o limitación semántica expresada como declaración lingüística en un determinado lenguaje textual.

En general, una restricción se puede unir a cualquier elemento o lista de elementos del modelo. Representa información semántica unida a un elemento del modelo, no sólo a una vista de él. Cada restricción tiene un cuerpo y un lenguaje de interpretación. El cuerpo es una cadena que codifica una expresión booleana para la condición en el lenguaje de la restricción. Una restricción se aplica a una lista ordenada de uno o más elementos del modelo. Observe que el lenguaje de especificación puede ser un lenguaje formal o puede ser lenguaje natural. En el último caso, la restricción será informal y no está sujeta a la ejecución automática (que no es decir que la ejecución automática sea siempre práctica para todos los lenguajes formales). UML proporciona el lenguaje de restricciones OCL [Warner-99], pero es posible usar otros lenguajes.

Algunas restricciones comunes tienen nombres para evitar escribir una declaración completa cada vez que son necesarias. Por ejemplo, la restricción **xor** entre dos asociaciones que compartan una clase común significa que un solo objeto de la clase compartida puede pertenecer a solamente una de las asociaciones al mismo tiempo.

Véase Capítulo 14, Elementos Estándar, para una lista de las restricciones predefinidas de UML.

Una restricción es una aserción, no un mecanismo ejecutable. Indica una condición que se debe hacer cumplir para el correcto diseño del sistema. Cómo garantizar una restricción es una decisión de diseño. Las restricciones de ejecución se hacen para ser evaluadas en los momentos en que el sistema instanciado es estable —es decir, entre la ejecución de operaciones y no en el medio de cualquier transacción atómica—. Durante la ejecución de una operación, puede haber momentos en que las restricciones se violen temporalmente.

Una restricción no se puede aplicar a sí misma.

Una restricción heredada —una restricción en un elemento antecesor del modelos o un estereotipo— debe seguirse aunque se definan restricciones adicionales en los descendientes. Una restricción heredada no puede ser pasada por alto o reemplazada. Si necesita hacerlo, el modelo está mal construido y debe ser reformulado. Sin embargo una restricción heredada se puede restringir, agregando restricciones adicionales. Si están en conflicto con las restricciones heredados por un elemento, entonces el modelo está mal formado.

Notación

Una restricción se representa como una cadena de texto entre llaves ({ }). La cadena de texto es el cuerpo codificado escrito en un lenguaje de restricción.

Es de esperar que las herramientas proporcionen uno o más lenguajes en los cuales se puedan escribir las restricciones formales. El lenguaje predefinido para la escritura de restricciones es OCL. Dependiendo del modelo, un lenguaje de programación tal como C++ puede ser útil para algunas restricciones. Si no, la restricción se puede escribir en lenguaje natural, con responsabilidad humana cara a la interpretación y ejecución. El lenguaje de cada restricción es parte de la restricción en sí misma, aunque el lenguaje no se expone generalmente en el diagrama (la herramienta no lo pierde de vista).

Para una lista de los elementos representados por las cadenas de texto en un compartimento (tal como los atributos dentro de una clase): una cadena de restricción puede aparecer como una entrada en la lista (Figura 13.167). La entrada no representa un elemento del modelo. Es una *restricción viva* que se aplica a todos los elementos correctos de la lista hasta otro elemento de la lista de restricciones o el fin de la lista. La restricción viva se puede sustituir por otra restricción viva más adelante en la lista. Para eliminar una restricción de ejecución, la sustituimos por una restricción vacía. Una restricción unida a un elemento individual de la lista no sustituye la restricción viva, pero puede aumentarla con restricciones adicionales.

Para un solo símbolo gráfico (tal como una clase o una línea de asociación), la cadena de restricción se puede colocar cerca del símbolo, preferiblemente cerca del nombre del símbolo, si lo hay.

Transacción CA	
{ valor $\geq$ 0 }	restricción de ejecución
cantidad: Dinero { valor es múltiplo de 20\$ }	restricción individual y restricción viva
saldo: Dinero	sólo se aplica la restricción de ejecución

Figura 13.167 Restricciones dentro de listas

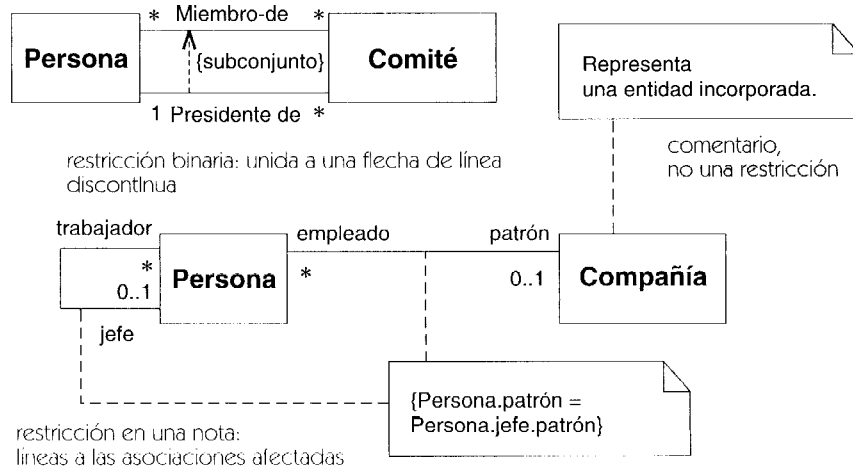


Figura 13.168 Notación de restricciones

Para dos símbolos gráficos (tales como dos clases o dos asociaciones): la restricción se representa como una flecha con línea discontinua de un elemento al otro elemento etiquetado por la cadena de la restricción (entre llaves). La dirección de la flecha es información relevante dentro de la restricción (Figura 13.168).

Para tres o más símbolos gráficos: La cadena de la restricción se coloca en un símbolo de nota y es unida a cada símbolo por una línea discontinua (Figura 13.168). Esta notación se puede utilizar para otros casos también. Para tres o más líneas del mismo tipo (tales como líneas de generalización o líneas de asociación), la restricción se puede unir a una línea discontinua que atraviesa todas las líneas. En caso de ambigüedad, las diferentes líneas se pueden numerar o etiquetar para establecer su correspondencia con la restricción.

Discusión

Una restricción hace una declaración semántica sobre el modelo en sí mismo, mientras que un comentario es un texto sin fuerza semántica y se puede unir a un elemento del modelo o a un elemento de representación. Ambos, restricciones y comentarios se pueden representar usando notas. En principio, las restricciones son ejecutables por las herramientas. En la práctica, algunas pueden ser difíciles de indicar formalmente y pueden requerir la ejecución humana. En el amplio sentido de la palabra, muchos elementos en un modelo son restricciones, pero la palabra restricción se utiliza para indicar las declaraciones semánticas que son difíciles de expresar con los elementos incorporados al modelo y que se deben indicar de manera lingüística.

Las restricciones se pueden expresar en cualquier lenguaje conveniente o aun en un lenguaje humano, aunque una restricción en lenguaje humano no se puede verificar por una herramienta.

El lenguaje OCL [Warmer-99] está diseñado para especificar restricciones de UML, pero bajo algunas circunstancias puede ser más apropiado un lenguaje de programación.

Ya que las restricciones se expresan como cadenas de texto, una herramienta de modelado genérica puede admitirlas y mantener su significado sin entenderlo. Por supuesto, una herra-

mienta o un dispositivo suplementario que verifique o haga cumplir la restricción debe entender la sintaxis y la semántica del lenguaje.

Se puede unir una lista de restricciones a la definición de un estereotipo. Esto indica que todos los elementos que llevan el estereotipo están conforme a la restricción.

Ejecución. Cuando el modelo contiene restricciones, no es necesario especificar qué hacer si son violadas. Un modelo es una declaración de lo que se supone que debe suceder. El hacer que suceda es trabajo de la implementación. Un programa puede contener aserciones y otros mecanismos de la validación, pero una falta contra una restricción se debe considerar una falta de programación. Por supuesto, si un modelo puede ayudar a producir un programa que esté listo para la construcción o se pueda verificar como correcto, entonces ha respondido a su propósito.

Elementos estándar

invariant, postcondition, precondition.

reunificación

Un lugar de una máquina de estados, diagrama de actividades o diagrama de secuencia en el cual se unen dos o mas rutas de control alternativas; una “desbifurcación”. Antónimo: bifurcación.

Véase también estado de unión o conjunción.

Semántica

Una reunificación no es otra cosa que una situación en la que se juntan dos o más rutas de control. En una máquina de estados, un estado tiene más de una transición de entrada. No se requiere ni se proporciona ninguna estructura especial del modelo para indicar una reunificación. Se puede indicar mediante un estado de unión si forma parte de una única ruta que se ejecuta hasta finalizar.

Notación

Una reunificación puede aparecer en un diagrama de estados, un diagrama de actividades o un diagrama de secuencia como un rombo con dos o más transiciones de entrada y una sola transición de salida. No se necesitan condiciones. La Figura 13.169 muestra un ejemplo.

También se emplea un rombo para las bifurcaciones (que son la inversa de las fusiones), pero la bifurcación se distingue claramente porque posee una transición de entrada y múltiples transiciones de salida, cada una de las cuales posee su propia condición de guarda.

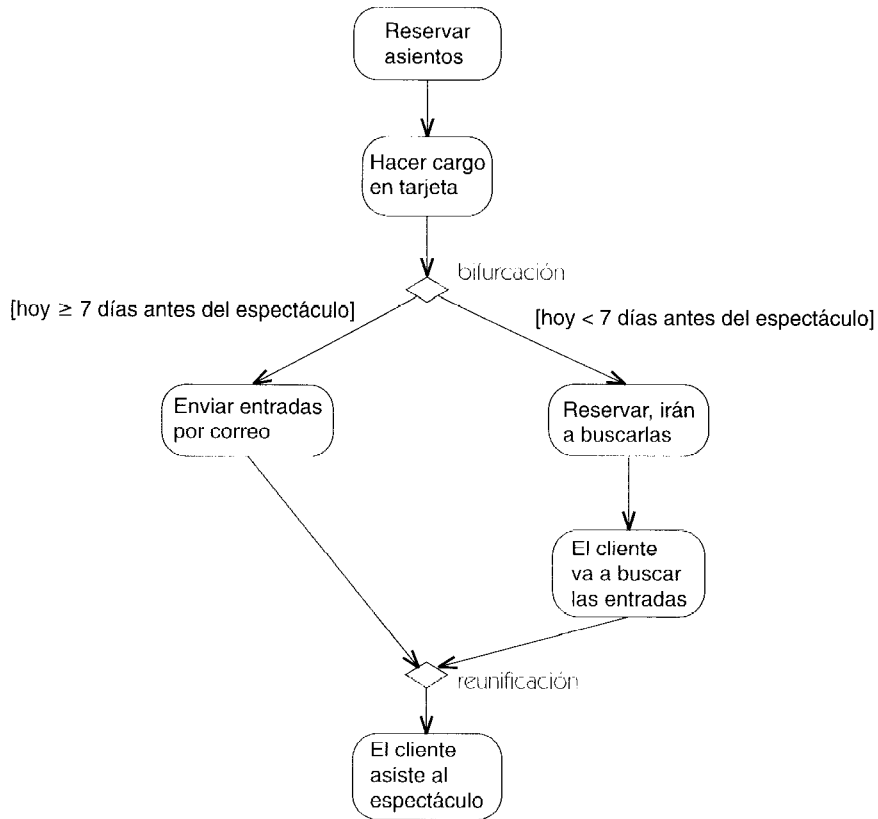


Figura 13.169 Reunificación

Una combinación de bifurcación y reunificación es válida pero tiene una utilidad limitada. Tendría múltiples transiciones de entrada y múltiples transiciones de salida con etiqueta.

Obsérvese que una reunificación es tan sólo una forma cómoda de notación y se puede omitir sin pérdida de información. Normalmente, las bifurcaciones y las fusiones se emparejan de forma anidada.

Discusión

Asegúrese de que distingue las reunificaciones de las uniones. Las reunificaciones combinan dos o más rutas alternativas de control. En toda ejecución se seguirá solamente una ruta en cada momento. No se necesita una sincronización.

La unión combina dos o más rutas concurrentes de control. En toda ejecución se seguirán todas las rutas y la unión sólo se disparará cuando todas ellas hayan llegado a los estados de origen de la unión.

reutilización

Utilización de un artefacto existente previamente.

rol

Ranura con nombre situada dentro de la estructura de un objeto que representa el comportamiento de un elemento que participa en un contexto (por oposición a las cualidades inherentes del elemento en cualquiera de sus aplicaciones). Un rol puede ser estático (como un extremo de asociación) o dinámico (como un rol de colaboración). Entre los roles de colaboración se cuentan los roles de clasificador y los roles de asociación.

Véase colaboración.

rol de asociación

Conexión de dos roles de clasificador dentro de una colaboración. Asociación entre dos clasificadores que se aplica sólo a un cierto contexto especificado por una colaboración.

Véase también asociación, colaboración.

Semántica

Un rol de asociación es una asociación que está definida y tiene significado sólo en el contexto descrito por una colaboración. Es una relación que es parte de la colaboración pero no una relación inherente en otras situaciones. Los roles de asociación son la parte estructural clave de las colaboraciones, pues permiten describir relaciones contextuales.

Dentro de una colaboración, un rol de clasificador denota una aparición individual de un clasificador, distinta de otras apariciones del clasificador y distinta también de la propia declaración del clasificador. Es un clasificador por derecho propio que representa una restricción en el uso del clasificador base basada en el contexto de una colaboración. De forma similar, un rol de asociación representa una asociación utilizada en un contexto particular, a menudo un uso restringido de una asociación normal. Un rol de asociación conecta dos roles de clasificador. Cuando se instancia una colaboración, los objetos se enlazan a roles de clasificador y los enlaces a roles de asociación. Un objeto puede desempeñar (enlazarse a) más de un rol.

Un rol de asociación conecta dos o más clasificadores o roles de clasificador dentro de una colaboración. El rol de asociación tiene una referencia a la asociación —la asociación base— y puede tener una multiplicidad que indique cuántos enlaces pueden representar el rol en una instancia de la colaboración. En algunos casos, las conexiones dentro de la colaboración pueden verse como usos de una asociación general entre las clases participantes. La colaboración muestra una forma de utilizar la asociación general para un propósito determinado dentro de la colaboración.

En otros casos, los roles de clasificador están conectados mediante asociaciones que no tienen validez fuera de la colaboración. Si un rol de asociación no tiene una asociación base explícita, entonces define una asociación implícita válida únicamente dentro de la colaboración.

Notación

Un rol de asociación se representa de la misma forma que una asociación, en concreto, mediante una línea continua entre dos símbolos de rol de clasificador (véase Figura 13.170). Es fácil distinguir que se trata de un rol de asociación pues conecta roles de clasificador.

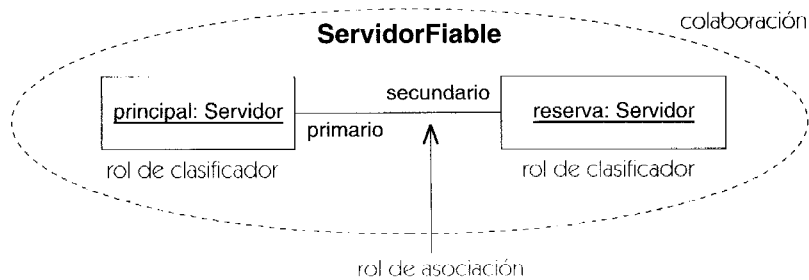


Figura 13.170 Rol de asociación

Elementos estándar

new, transient.

rol de clasificador

Ranura de una colaboración que describe el rol desempeñado por un participante en una colaboración.

Véase también colaboración.

Semántica

Una colaboración describe un patrón de interacción entre un conjunto de participantes, que son instancias de clases o de tipos de datos. Un rol de clasificador es la descripción de un participante. Cada rol es un uso distinto del clasificador en un contexto propio y único. Puede haber más de un rol para el mismo clasificador, cada uno de los cuales tendrá un conjunto distinto de relaciones con otros roles dentro de una colaboración. Sin embargo, un rol no es un objeto individual, sino una descripción de todos los objetos que pueden tomar parte en una instancia de una colaboración. En cada instancia de la colaboración, los roles serán desempeñados por diferentes objetos y enlaces.

Un rol de clasificador tiene una referencia a un clasificador (clasificador base) y una multiplicidad. El clasificador base restringe el tipo de objeto que puede desempeñar el rol de clasificador. La clase del objeto puede ser la misma o un descendiente del clasificador base. La multiplicidad indica cuántos objetos pueden desempeñar el rol a la vez en una instancia de la colaboración.

Un rol de clasificador puede tener un nombre o ser anónimo. Puede tener múltiples clasificadores base si se realiza clasificación múltiple.

Un rol de clasificador puede conectarse con otros roles de clasificador mediante roles de asociación.

Objetos. Una colaboración representa un grupo de objetos que trabajan juntos para llevar a cabo un objetivo. Un rol representa la parte de uno de los objetos (o un grupo de los objetos) que reali-

za dicho objetivo. Un objeto es una instancia directa o indirecta de la clase base de su rol. No todos los objetos de la clase base aparecen necesariamente en una colaboración de su rol, además es posible que objetos de la misma clase base desempeñen múltiples roles en la misma colaboración.

El mismo objeto puede desempeñar diferentes roles en diferentes colaboraciones. Una colaboración representa una faceta de un objeto, pudiendo combinar un mismo objeto físico diferentes facetas conectando implícitamente las colaboraciones en las que desempeña algún rol.

Notación

Un rol de clasificador se representa utilizando el símbolo de clasificador (un rectángulo) con un nombre de rol y de clasificador separados por dos puntos, esto es, nombreRol:ClaseBase. Sin embargo, un rol no es un objeto, sino un clasificador que describe muchos objetos que aparecen en diferentes instancias de la colaboración.

Se puede omitir el nombre del rol o el del clasificador, pero deben aparecer los dos puntos para distinguirlo de una clase ordinaria. Dentro de una colaboración existe un pequeño riesgo de confusión, pues todos los participantes son roles.

Un rol de clasificador puede mostrar un subconjunto de las características del clasificador, es decir, los atributos y operaciones utilizados en un contexto dado. El resto de características pueden suprimirse si no se utilizan.

La Figura 13.171 muestra diversas formas que puede tomar un rol de clasificador.

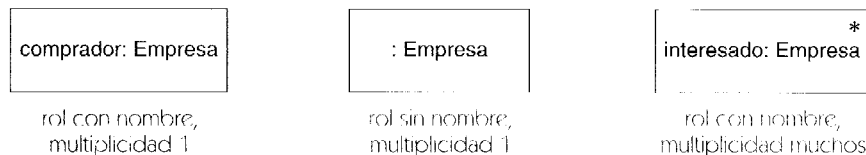


Figura 13.171 Rol de clasificador

Elementos estándar

destroyed, new, transient.

rol de colaboración

Ranura para un objeto o enlace dentro de una colaboración. Especifica el tipo de objeto o enlace que puede aparecer en una instancia de la colaboración.

Véase también colaboración, rol de asociación, rol de clasificador.

Semántica

Un rol de colaboración describe un objeto o un enlace. Sin embargo, no representa un único enlace u objeto sino un sitio que puede sustituirse por un objeto o un enlace cuando se

instancia la colaboración. Una colaboración es o bien un rol de clasificador o bien un rol de asociación. Un rol de clasificador tiene uno o más clasificadores base y puede ser instanciado como un objeto, el cual es una instancia de los clasificadores o uno de sus descendientes. Un rol de asociación puede tener una asociación base y ser instanciado como un enlace, el cual es una instancia de la asociación o uno de sus descendientes. En muchos casos, la asociación sólo está definida en la colaboración —esto es, ocurre sólo entre objetos que desempeñan los roles y no tiene significado fuera de la colaboración—. En estos casos se puede omitir la asociación base. La asociación se define implícitamente por su aparición en la colaboración, con la restricción de que no puede ser utilizada en otro sitio.

Notación

Los roles de colaboración pueden ser roles de clasificador o roles de asociación.

Rol de clasificador. Un rol de clasificador es un clasificador y se representa mediante el símbolo de las clases: un rectángulo. A menudo sólo se muestra el compartimento del nombre, que contiene la cadena:

```
nombreRolClasificador :NombreClasificador Baseinst.
```

El nombre de clasificador base puede incluir la ruta de paquetes completa si es necesario (una herramienta permitirá normalmente caminos abreviados cuando no haya ambigüedad). Los nombres de paquete preceden al nombre de la clase separados por dos signos de dos puntos, por ejemplo:

```
mostrar_ventana:SistemaVentanas::VentanasGraficas:: Ventana
```

Un estereotipo para un rol de clasificador puede representarse textualmente entre comillas dobles sobre la cadena del nombre o como un icono en la esquina superior derecha. El estereotipo de un rol de clasificador debe coincidir con el estereotipo de su clasificador base.

Un rol de clasificador que representa un conjunto de objetos incluye un indicador de multiplicidad (por ejemplo un asterisco) en la esquina superior derecha del símbolo de la clase. Este indicador especifica el número de objetos que pueden ligarse al rol en una instancia de la colaboración. Cuando se omite el indicador, el valor es exactamente uno.

El nombre del rol de clasificador puede omitirse, en cuyo caso deberían mantenerse los dos puntos al lado del nombre de la clase. Este caso representa un objeto anónimo de la clase.

Si hay clasificación múltiple, el rol puede tener más de un clasificador, en ese caso, el objeto es una instancia de cada uno de ellos.

La clase del rol de clasificador puede suprimirse (junto con los dos puntos).

Un objeto o enlace que se crea durante una interacción tiene la palabra clave **new** como restricción en su rol. Un objeto o enlace destruido durante una interacción tiene la palabra clave **destroyed** como restricción en su rol. Las palabras clave pueden utilizarse incluso aunque el elemento no tenga nombre. Ambas palabras clave pueden utilizarse conjuntamente, pero resulta más útil emplear la palabra clave **transient** en lugar de la combinación **new destroyed**.

Rol de asociación. Un rol de asociación es una asociación, y se representa mediante una ruta entre dos símbolos de rol de clasificador. La ruta puede tener asociado un nombre de la forma:

```
nombreRolAsociación NombreAsociaciónBase
```

Si se omite el nombre de la asociación base, entonces no hay asociación base. En la ruta pueden mostrarse los nombres de rol y otros adornos de la asociación base.

Si un extremo de la ruta del rol de asociación está conectado a un rol de clase con una multiplicidad distinta de uno, puede situarse un indicador de multiplicidad en ese extremo de la asociación para enfatizar dicha multiplicidad.

La Figura 13.46 muestra un ejemplo de roles de colaboración.

ruta

Se trata de una serie conexas de segmentos gráficos que conectan símbolos entre sí, normalmente para mostrar una relación.

Notación

Las *rutas* son conexiones gráficas entre los símbolos de un diagrama. Las rutas se utilizan en esta notación para las relaciones de distintas clases, tales como asociaciones, generalizaciones y dependencias. Los extremos de dos segmentos conectados coinciden. Un segmento puede ser un segmento de línea recta, un arco o alguna otra forma, aun cuando muchas herramientas admiten únicamente líneas rectas y arcos (Figura 13.172). Las líneas se pueden dibujar en cualquier ángulo aunque algunos creadores de modelos prefieren limitar las líneas a ángulos rectos y posiblemente impongan una cuadrícula rectangular por razones estéticas y para facilitar el trazado. En general, el trazado de una ruta no tiene significado, aunque las rutas deberían evitar cruzar regiones cerradas, porque cruzar los límites de una región gráfica puede tener significado semántico. (Por ejemplo, una asociación entre dos clases de una colaboración debería dibujarse dentro de la región de colaboración para indicar una asociación entre objetos de la misma instancia de colaboración; sin embargo, una ruta que hiciera una salida de la región indicaría una asociación entre objetos procedentes de distintas instancias de colaboración.) Más exactamente, las rutas son topológicas. Su trazado exacto no tiene semántica, pero su conexión con otros símbolos y sus intersecciones poseen un significado. El trazado exacto de las rutas tiene gran importancia a efectos de la compresión y de la estética, por supuesto; puede connotar de forma sutil la importancia de las relaciones y otras cosas. Pero estas consideraciones son para los seres humanos y no para los computadores. Se supone que las herramientas admitirán de forma sencilla los trazados y retrazados de las rutas.

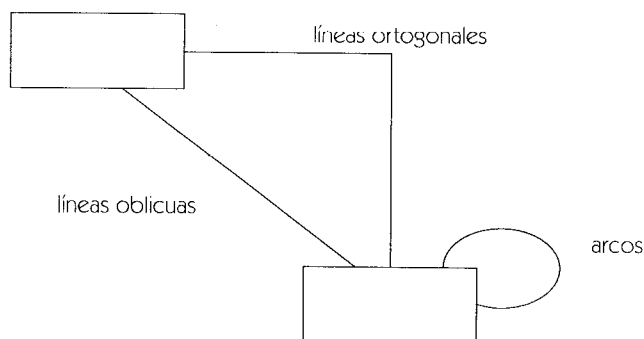


Figura 13.172 Rutas

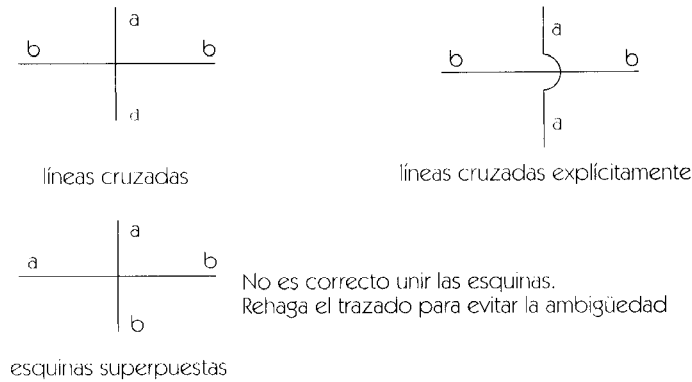


Figura 13.173 Cruces de rutas

En la mayoría de los diagramas, los cruces de líneas carecen de significación. Para evitar la ambigüedad respecto a la identidad de las líneas que se cruzan, se puede dibujar un pequeño semicírculo o discontinuidad en una de las líneas en el punto de cruce (Figura 13.173). Con mayor frecuencia, los creadores de modelos se limitan a tratar el cruce como dos líneas independientes y están de acuerdo en evitar la confusión que surgiría al hacer que dos ángulos rectos se tocaran en el vértice.

En algunas relaciones (tales como la agregación y la generalización), varias rutas del mismo tipo se pueden conectar a un mismo símbolo. Si las propiedades de los distintos elementos del modelo coinciden, entonces los segmentos de línea conectados con el símbolo se pueden combinar en un único segmento de línea, de tal modo que la ruta que sale del símbolo se ramifique formando una especie de árbol (Figura 13.174). Esto no pasa de ser una opción de

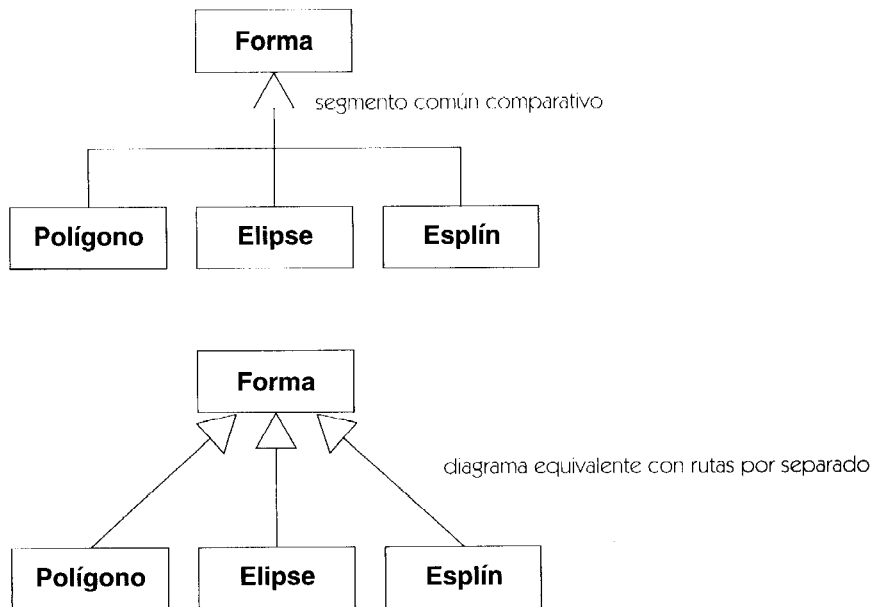


Figura 13.174 Rutas con segmento compartido

representación gráfica. Conceptualmente, las rutas individuales son diferentes. Esta opción de representación no puede utilizarse cuando la información de modelado de los diferentes segmentos no es idéntica.

secuencia de acciones

Sucesión de acciones que se ejecutan una detrás de otra. Es un tipo de acción.

Semántica

Una secuencia de acciones es una lista de acciones. Las acciones se ejecutan secuencialmente. La secuencia completa se considera una unidad atómica (es decir, no puede ser interrumpida). Una secuencia de acciones es una acción y puede por consiguiente ser asociada a transiciones o a pasos de interacción.

Notación

Una secuencia de acciones se representa mediante una cadena formada por una secuencia de cadenas separadas por signos de punto y coma.

Si las acciones se expresan en un determinado lenguaje de programación, se puede utilizar la sintaxis sobre secuencia de acciones del lenguaje en lugar de la representación descrita.

Ejemplo

```
contador:=0;reservas.borrar();send Quiosco.primeraPantalla()
```

semántica

Especificación formal del significado y comportamiento de algo.

señal

Especificación de una comunicación asíncrona entre objetos. Las señales pueden tener parámetros que se expresan como atributos.

Véase también evento, mensaje, enviar.

Semántica

Una señal es un clasificador explícito con nombre, destinado a una comunicación explícita entre objetos. Posee una lista explícita de parámetros, representados como sus atributos. Es enviada

explícitamente por un objeto a otro objeto o conjunto de objetos. La emisión general de una señal se puede considerar como enviar una señal al conjunto de todos los objetos —aunque en la práctica se implementaría de otra manera para mayor eficiencia—. El emisor especifica los argumentos de la señal en el momento en que se envía ésta. Enviar una señal es equivalente a instanciar un objeto señal y transmitir ese objeto al conjunto de objetos destino. La recepción de una señal es un evento que está destinado a disparar transiciones en la máquina de estados del receptor. Una señal que se envía a un conjunto de objetos puede disparar cero o una transiciones en cada objeto independientemente. Las señales son medios explícitos mediante los cuales los objetos se pueden comunicar entre sí asíncronamente. Para efectuar una comunicación síncrona, es preciso utilizar dos señales, una en cada dirección.

Las señales son elementos generalizables. Una señal hija se deriva de una señal padre. Hereda los atributos del padre y puede agregar sus propios atributos adicionales. Una señal hija dispara una transición que haya sido declarada para utilizar una de sus señales antecesoras.

La declaración de una señal tiene como alcance el paquete en que haya sido declarada. No está restringida a una única clase.

Una clase o una interfaz pueden declarar las señales que están dispuestas a manejar. Esta declaración es una recepción, que puede incluir la especificación de los resultados que se esperan cuando se reciba la señal. La declaración posee una propiedad que indica si es polimórfica. Si lo es, entonces una clase descendiente podrá manejar la señal, evitando quizá que la señal llegue a la clase actual. Si no es polimórfica, entonces ningún descendiente podrá interceptar el manejo de la señal.

Una señal es un clasificador y puede tener operaciones que accedan a sus atributos y los modifiquen. Además todas las señales comparten la operación implícita `send`.

`send (conjuntoDestino)`

La señal se envía a todos los objetos del conjunto destino.

Notación

La palabra reservada de estereotipo «**signal**» se pone delante de la declaración de aquellos parámetros que tengan el nombre de una señal, para indicar que hay una clase o interfaz que admite la señal. Los parámetros de la señal se incluyen en la declaración. La declaración no puede tener un tipo de retorno.

La declaración de una señal se puede expresar como un estereotipo de un símbolo de clase. La palabra reservada «**signal**» aparece en un rectángulo por encima del nombre de la señal. Los parámetros de la señal aparecen como atributos dentro del compartimento de atributos. El compartimento de operaciones puede contener operaciones de acceso.

La Figura 13.175 muestra la utilización de una notación de generalización para relacionar una señal hija con su padre. La hija hereda los parámetros de sus antecesores y puede agregar sus propios parámetros adicionales. Por ejemplo, la señal **BotonRatonPulsado** tiene los atributos **tiempo**, **dispositivo** y **localización**.

Para utilizar una señal como disparador de una transición, utilice la sintaxis:

`nombre-evento ( parámetroslista )`

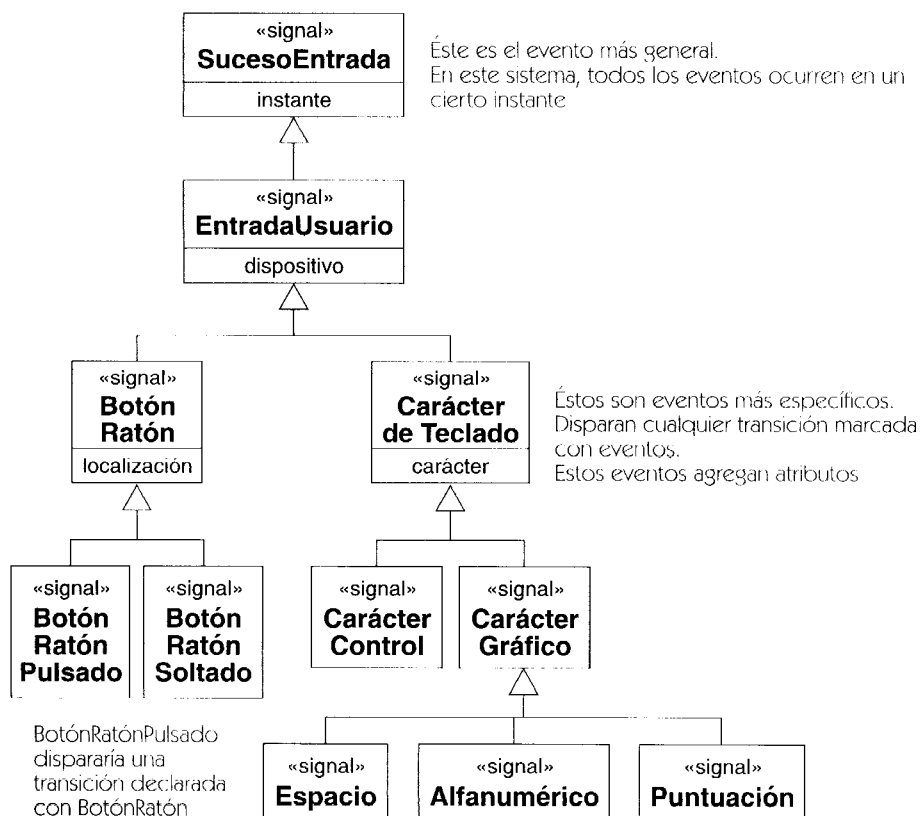


Figura 13.175 Declaraciones de señal

Los parámetros tienen la sintaxis:

nombre-parámetro :expresión-tipo

Los parámetros de una señal se declaran como atributos con valor inicial, que se puede anular durante la iniciación o al enviar. El valor inicial se usa si se crea una instancia de señal, se inicia y después se envía a un objeto. Si se envía una señal empleando una sintaxis de llamada a operación, entonces los valores iniciales son los valores por defecto de los parámetros de la señal.

Discusión

Una señal es la más fundamental comunicación entre objetos; posee una semántica más sencilla y más clara que las llamadas a procedimientos. Una señal es inherentemente a una comunicación asíncrona unidireccional que va de un objeto a otro y en la cual se pasa por valor toda la información. Es un modelo adecuado para la comunicación en sistemas concurrentes y distribuidos.

Para construir una comunicación asíncrona, utilice parejas de señales (una en cada dirección). Una llamada se puede ver como una señal con un parámetro implícito que es un puntero de retorno.

signatura

Nombre y propiedades de los parámetros de una característica de comportamiento, tal como una señal u operación. La signatura puede incluir tipos de retorno opcionales (para las operaciones, no para las señales).

Semántica

La signatura de una operación forma parte de su declaración. Parte de la signatura (pero no toda) se emplea para verificar la equivalencia de operaciones y métodos, con objeto de evitar conflictos o anulaciones. Para este propósito, se incluye el nombre de la operación y la lista ordenada de los tipos de los parámetros, pero no sus nombres ni direcciones y se excluyen los parámetros de retorno. Si coinciden dos signaturas pero las propiedades restantes son inconsistentes (por ejemplo, si un parámetro de entrada corresponde a uno de salida) entonces las declaraciones están en conflicto y el modelo está mal formado.

sistema

Colección de unidades conectadas que se organiza para lograr un propósito. Un sistema puede estar descrito por uno o más modelos, posiblemente desde distintos puntos de vista. El sistema es el “modelo completo”.

Semántica

El sistema se modela mediante un subsistema de nivel máximo que contiene indirectamente todo el conjunto de elementos del modelo que funcionan como un todo para lograr algún propósito del mundo real.

Notación

Se puede mostrar el sistema como un paquete con el estereotipo «**system**». Sin embargo, no suele ser necesario mostrar el sistema como una unidad.

solicitud

Especificación de un estímulo que se envía a las instancias. Puede ser la llamada a una operación o bien el envío de una señal.

subclase

Hija de otra clase en una relación de generalización —esto es, la descripción más específica—. La clase hija se denomina subclase. La clase padre se denomina superclase.

Véase generalización, herencia.

Semántica

Una subclase hereda la estructura, relaciones y comportamiento de su superclase; además puede hacer sus propias adiciones.

subestado

Estado que forma parte de un estado compuesto.

Véase estado compuesto, subestado concurrente, subestado disjunto.

subestado concurrente

Un subestado que se puede llevar a cabo simultáneamente con otros subestados contenidos en el mismo estado compuesto.

Véase estado compuesto, subestado disjunto.

subestado disjunto

Un subestado que no se puede llevar a cabo simultáneamente con otros subestados contenidos en el mismo estado compuesto. Contraste: subestado concurrente.

Véase transición compleja, estado compuesto.

subestado ortogonal

Se trata de uno elemento de un conjunto de estados que descomponen un estado compuesto en subestados, todos los cuales están activados concurrentemente.

Véase estado compuesto, subestado concurrente.

submáquina

Máquina de estados que se puede invocar como parte de otra máquina de estados. Posee una semántica tal como si se hubiera duplicado su contenido para luego insertarlo en el estado al que hace referencia.

Véase estado, máquina de estados, estado de referencia a submáquina.

subsistema

Paquete de elementos que se tratan como una unidad, incluyendo una especificación de todo el contenido del paquete, que se trata como una unidad coherente. Un subsistema se modela

simultáneamente como paquete y como clase. Los subsistemas tienen un conjunto de interfaces que describen su relación con el resto del sistema y las circunstancias en que se puede utilizar.

Véase también interfaz, paquete, realización.

Semántica

Un subsistema es una pieza coherente de un sistema que se puede tratar como unidad abstracta. Representa el comportamiento emergente de una parte del sistema. Como unidad, posee su propia especificación de comportamiento y su parte de implementación. La especificación de comportamiento define su comportamiento emergente como unidad que puede interactuar con otros subsistemas. La especificación de comportamiento se da en términos de caso de uso y otros elementos de comportamiento. La parte de implementación describe la implementación del comportamiento en términos de los elementos subordinados de que consta su contenido y se da como un conjunto de colaboraciones entre los elementos contenidos.

El sistema en sí constituye el subsistema de más alto nivel. La implementación de un subsistema se puede escribir como una colaboración de subsistemas de nivel inferior. De esta manera, se puede expandir todo el sistema como un jerarquía de subsistemas hasta definir los subsistemas del más bajo nivel en términos de clases ordinarias.

Los subsistemas pueden incluir elementos estructurales y elementos de especificación, tales como casos de uso y operaciones exportadas por el subsistema. Las especificaciones del subsistema se implementan mediante elementos estructurales. El comportamiento del subsistema es el comportamiento de los elementos que contiene.

Estructura

La especificación de un subsistema abarca los elementos indicados como elementos de especificación, junto con las operaciones e interfaces que se hayan definido para el subsistema como un todo. Entre los elementos de especificación se cuentan los casos de uso, las restricciones, las relaciones entre casos de uso y demás. Estos elementos y operaciones definen el comportamiento del subsistema como entidad emergente, que es el resultado neto de la actuación conjunta de sus partes. Los casos de uso especifican secuencias completas de interacciones del subsistema con actores externos. Las interfaces especifican operaciones que deban proporcionar el subsistema o sus casos de uso. La especificación no revela cuáles son las partes ni cómo interaccionan esas partes para realizar el comportamiento necesario.

El resto de los elementos del subsistema se encarga de dar vida a su comportamiento. Esto puede incluir distintas clases de clasificadores y las relaciones existentes entre ellos. Hay un conjunto de colaboraciones entre elementos de realización del subsistema que se encarga de dar vida a las especificaciones. En general, cada caso de uso requiere una o más colaboraciones. Cada colaboración describe la forma en que cooperan las instancias de los elementos de implementación para realizar conjuntamente el comportamiento especificado por un caso de uso u operación. Todos los mensajes entrantes y salientes en el subsistema en el nivel de especificación tienen que corresponderse con mensajes entre sus elementos de implementación y los de otros subsistemas.

Un subsistema es un paquete y tiene las propiedades de un paquete. En particular, la importación de subsistemas funciona tal como se ha descrito para los paquetes y la generalización entre subsistemas tiene las mismas consecuencias para la visibilidad de su contenido.

Notación

Los subsistemas se denotan con un símbolo de paquete (un rectángulo con una pequeña ficha) que contiene la palabra reservada «**subsystem**» por encima del nombre del subsistema (Figura 13.176).

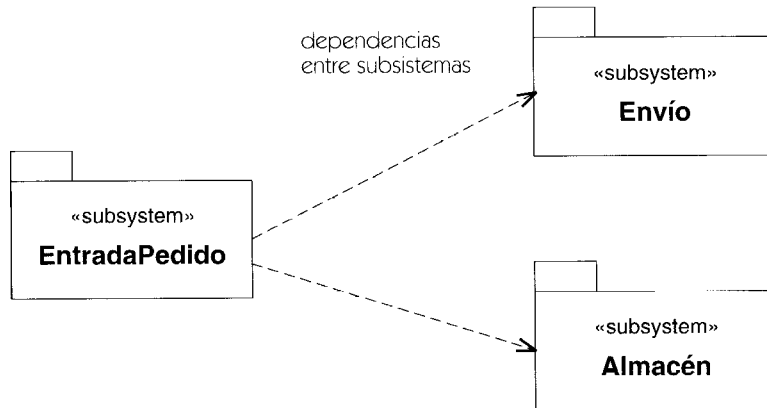


Figura 13.176 Subsistemas

Discusión

Un subsistema es un grupo emergente de elementos de diseño, tales como clases lógicas. Un componente es un grupo emergente de elementos de implementación, tales como clases del nivel de implementación. En muchos casos, los subsistemas se implementan como componentes. Esto simplifica la correspondencia entre diseño e implementación; por tanto, se trata de un enfoque corriente para la arquitectura. Además, muchos componentes se implementan como clases dominantes que implementan directamente las interfaces de los componentes. En este caso, un subsistema, un componente y una clase podrían tener todos ellos la misma interfaz.

subtipo

Tipo que es hijo de otro tipo. Se puede utilizar el término hijo, más neutro, para cualquier elemento generalizable

Véase generalización.

superclase

Padre de otra clase en una relación de generalización, esto es, la especificación de elemento más general. La clase hija se denomina subclase. La clase padre se denomina superclase.

Véase generalización.

Semántica

Las subclases heredan la estructura, relaciones y comportamiento de sus superclases y pueden hacer adiciones.

supertipo

Sinónimo de superclase. Se puede emplear el término más neutro, padre, para cualquier elemento generalizable.

Véase generalización.

supratipo

Denota una metaclass cuyas instancias son subclases de una clase dada.

Véase también metaclass.

Semántica

Las subclases de una clase dada pueden considerarse a su vez instancias de una metaclass. Esta metaclass es lo que se denomina un supratipo. Por ejemplo, la clase **Árbol** puede tener las subclases **Roble**, **Olmo** y **Sauce**. Consideradas como objetos, estas subclases son instancias de la metaclass **EspecieDeÁrbol**. Entonces **EspecieDeÁrbol** es un supratipo de nivel superior a **Árbol**.

Notación

Un supratipo se representa mediante una clase con el estereotipo «**powertype**». Está conectado a un conjunto de rutas de generalización mediante una flecha discontinua que tiene el estereotipo «**powertype**» asociado a la flecha. (Figura 13.177).

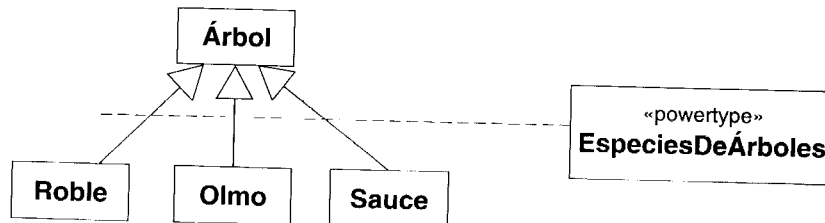


Figura 13.177 Supratipo

tiempo

Valor que representa un instante de tiempo absoluto o relativo.

Véase expresión temporal.

tiempo de análisis

Tiempo durante el cual se realiza la actividad de análisis en un proceso de desarrollo de software. No suponga que todo el análisis de un sistema se realiza al mismo tiempo o que precede a otras actividades como el diseño o la implementación, ya que las diversas actividades pueden ser secuenciales para un determinado elemento, pero cuando se trata del sistema completo las actividades se entremezclan.

Véase tiempo de diseño, tiempo de modelado.

tiempo de compilación

Se refiere a algo que ocurre durante la compilación de un módulo de software.

Véase tiempo de ejecución, tiempo de modelado.

tiempo de ejecución

Período de tiempo durante el cual se ejecuta un programa de computador. Contrastar con: tiempo de modelado.

tiempo de modelado

Denota todo aquello que se produce durante una actividad de modelado correspondiente al proceso de desarrollo del software. Se incluye el análisis y el diseño. Nota de utilización: cuando se discuten sistemas de objetos, suele ser importante distinguir entre los problemas del tiempo de modelado y del tiempo de ejecución.

Véase también proceso de desarrollo, fases de modelado.

tiempo de transición

Véase marca de tiempo.

tiempo de diseño

Se refiere a lo que ocurre durante la actividad de diseño del proceso de desarrollo de software. Contrastar con: tiempo de análisis.

Véase tiempo de modelado, fases de modelado.

tipo

Como adjetivo: El clasificador declarado que tienen que contener el valor de un atributo, parámetro o variable. El valor real tiene que ser una instancia del tipo o de uno de sus descendientes.

Como sustantivo: Estereotipo de una clase que se emplea para especificar un conjunto de instancias (tales como objetos) junto con las operaciones aplicables a esos objetos. Un tipo no puede contener métodos. Contrastar con: interfaz, clase de implementación.

Tipo y clase de implementación

Las clases se pueden estereotipar como tipos o clases de implementación (aunque también pueden quedar sin ser diferenciadas). Los tipos se emplean para especificar un dominio de objetos, junto con los parámetros aplicables a los objetos, sin definir la implementación física de los objetos o de sus operaciones. Los tipos no pueden contener métodos pero pueden proporcionar especificaciones de comportamiento para sus operaciones. También pueden incluir atributos y asociaciones para especificar el comportamiento de sus operaciones. Los atributos y asociaciones de un tipo no determinan la implementación de sus objetos.

Una clase de implementación define la estructura física de los datos y métodos de sus objetos, tal como se implementan en lenguajes tradicionales como C++ y SmallTalk. Se dice que la clase de implementación realiza un tipo si la clase de implementación incluye todas las operaciones del tipo con el mismo comportamiento. Una clase de implementación puede realizar múltiples tipos y múltiples clases de implementación pueden realizar un mismo tipo. Los atributos y asociaciones de una clase de implementación no tienen por qué ser los mismos que los de un tipo que realice. La clase de implementación puede proporcionar métodos para sus operaciones en términos de sus atributos y asociaciones físicas y puede declarar operaciones adicionales que no se encuentren en ningún tipo.

Un objeto sólo puede ser instancia de una clase de implementación, que especifica la implementación física del objeto. Sin embargo, un objeto puede ser instancia de múltiples tipos. Si el objeto es una instancia de una clase de implementación, entonces la clase de implementación tiene que realizar los tipos de los cuales sea instancia el objeto. Si se dispone de clasificación dinámica, entonces un objeto puede ganar y perder tipos a lo largo de su ciclo de vida. Los tipos que se emplean de este modo caracterizan el rol cambiante que puede adoptar un objeto para luego abandonarlo.

Aun cuando el uso de los tipos y de las clases de implementación es diferente, su estructura interna es la misma, así que se modelan como estereotipos de clases. Admiten la generalización en toda su extensión, así como el principio de capacidad de sustitución y la herencia de atributos, asociaciones y operaciones. Sin embargo, los tipos sólo pueden especializar a otros tipos y las clases de implementación sólo pueden especializar a otras clases de implementación. Los tipos sólo pueden estar relacionados con clases de implementación por realización.

Notación

Las clases sin diferenciar se muestran sin palabra reservada. Los tipos se muestran con la palabra clave «**type**» y las clases de implementación se muestran con la palabra clave «**clase de implementación**». La Figura 13.178 muestra un ejemplo.

La implementación de un tipo por parte de una clase de implementación se modela empleando la relación de realización, que se denota mediante una línea discontinua con una punta de flecha triangular rellena (una “flecha discontinua de generalización” o una “dependencia con

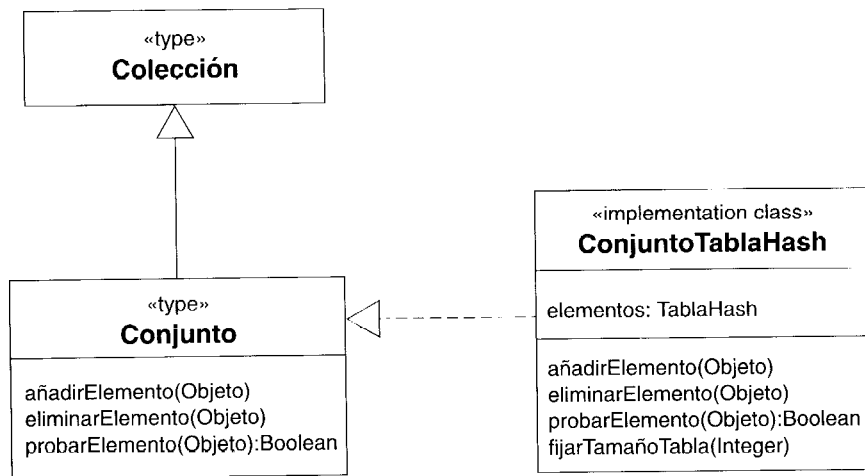


Figura 13.178 Notación para los tipos y clases de implementación

punta de flecha rellena”). Este símbolo implica la herencia de operaciones pero no de estructura (atributos o asociaciones). La realización puede emplearse entre dos clasificadores cualesquiera para los cuales uno admita las operaciones del otro, pero la forma habitual en que se usa es la implementación de un tipo por parte de una clase de implementación.

Discusión

Los tipos y las clases de implementación son conceptos relativamente limitados, destinados a los lenguajes de programación convencionales como C++ y SmallTalk. El concepto de clase de UML se puede emplear directamente de una forma más general. UML admite tanto la clasificación múltiple como la clasificación dinámica, lo cual elimina la necesidad de distinguir entre tipos y clases de implementación. Las diferencias, sin embargo, son útiles para diseñar código destinado a lenguajes convencionales.

tipo de dato

Un descriptor de un conjunto de valores que carecen de identidad (existencia independiente y la posibilidad de efectos secundarios). Los tipos de datos incluyen tipos primitivos predefinidos y tipos definibles por el usuario. Los tipos primitivos son números, cadenas, y tiempo.

Los definidos por el usuario son enumeraciones. Los tipos anónimos de datos previstos para la implementación en un lenguaje de programación se pueden definir usando tipos del lenguaje.

Véase también clasificador, identidad.

Semántica

Los tipos de datos son los primitivos predefinidos que se necesitan como base de los tipos definidos por el usuario. Su semántica se define matemáticamente fuera de los mecanismos de

construcción de tipos de un lenguaje. Los números están predefinidos. Incluyen números enteros y reales. También están predefinidas las cadenas. Estos tipos de datos no son definidos por el usuario.

Los tipos enumeración son conjuntos finitos, definidos por el usuario, de elementos nombrados que tienen un orden definido entre sí mismos y ninguna otra propiedad computacional. Un tipo enumeración tiene un nombre y una lista de las constantes de la enumeración. El tipo **booleano** se define con los literales de la enumeración **false** y **true**.

Las operaciones se pueden definir en los tipos de datos, y las operaciones pueden tener tipos de datos como parámetros. Debido a que un tipo de datos no tiene ninguna identidad y es sólo un valor, las operaciones en los tipos de datos no los modifican; en su lugar, simplemente vuelven valores. No tiene ningún sentido hablar de crear un nuevo valor del tipo de datos, porque carece de identidad. Todos los valores del tipo de datos se encuentran (conceptualmente) predefinidos. Una operación en un tipo de datos es una consulta que puede no cambiar el estado del sistema pero puede retornar un valor.

Un tipo de dato se puede también describir por un tipo del lenguaje —una expresión de un tipo de datos en un lenguaje de programación—. Dicha expresión señala un tipo de dato anónimo en el lenguaje en cuestión. Por ejemplo, la expresión **Persona* (*) (String)** denota una expresión de tipo en C++ que no se corresponde a ningún tipo simple de datos con un nombre.

tipo del lenguaje

Dícese de un tipo de datos anónimo que se define en la sintaxis de un lenguaje de programación.

Véase también tipo de dato.

Semántica

Un tipo del lenguaje es una expresión que debe ser interpretada como un tipo de dato de algún lenguaje de programación. Se puede emplear como tipo de un atributo, variable o parámetro. No posee nombre y no declara un nuevo tipo de dato.

Por ejemplo, el tipo de dato de C++ "**Persona* (*) (Contrato*,int)**" se podría definir como un tipo del lenguaje C++.

La intención de un tipo del lenguaje es la implementación en un determinado lenguaje de programación. Para relaciones más lógicas deben emplearse asociaciones.

tipo primitivo

Este término denota un tipo de dato básico predefinido, tal como un entero o una cadena.

Véase también enumeración.

Semántica

Las instancias de los tipos primitivos carecen de identidad. Si dos instancias tienen la misma representación, entonces son indistinguibles y se pueden pasar por valor sin pérdida de información.

Entre los tipos primitivos se cuentan los números y las cadenas, así como posiblemente otros tipos de datos dependientes del sistema tales como fechas y valores monetarios, cuya semántica está predefinida fuera de UML.

Se supone que los tipos primitivos tendrán una fuerte correspondencia con los que se hallan en el lenguaje de programación que se tenga como destino.

Véase también enumeraciones, que son tipos de datos definibles por el usuario y que no son tipos primitivos predefinidos.

transición

Relación entre dos estados dentro de una máquina de estados que indica que un objeto del primer estado va a ejecutar las acciones especificadas para después pasar al segundo estado cuando se produzca un evento especificado y se cumplan las condiciones de guarda. En este cambio de estado se dice que se dispara la transición. Las transiciones simples poseen un único estado de origen y un único estado blanco. Una transición compleja posee más de un origen y/o más de un estado destino. Representa un cambio en el número de estados activos concurrentemente o una bifurcación o reunificación de control. Las transiciones internas poseen estado de origen pero no estado destino. Representan la respuesta a un evento sin cambio de estado. Los estados y las transiciones son los vértices y nodos de las máquinas de estados.

Véase también máquina de estados.

Semántica

Las transiciones representan rutas potenciales entre estados dentro de la historia de los objetos, así como las acciones que se llevan a cabo al cambiar de estado. Una transición indica la forma en que responde a un evento un objeto que está en un cierto estado. Los estados y las transiciones son los vértices y arcos de una máquina de estados que describe las posibles historias de las instancias de un clasificador.

Estructura

Toda transición puede poseer un estado de origen, un evento disparador, una condición de guarda, una acción y un estado destino. Las transiciones pueden carecer de alguno de estos elementos.

Estado de origen. El estado de origen es el estado que se ve afectado por la transición. Si un objeto se encuentra en el estado de origen, se puede disparar una transición saliente si el objeto recibe el evento disparador de la transición y se cumple la condición de guarda (si existe).

Estado destino. El estado destino es el estado que está activo una vez finalizada la transición. Es el estado al que pasa el objeto maestro. El estado destino no se usa en las transiciones internas, que no llevan a cabo cambios de estado.

Evento disparador. El evento disparador es el evento cuya recepción por parte del objeto cuando se encuentra en el estado de origen hace que la transición pueda dispararse, siempre y cuando se cumpla la condición de guarda. Si el evento posee parámetros, sus valores estarán disponibles para la transición y se pueden utilizar en expresiones para la condición de guarda y en acciones. El evento que dispara una transición pasa a ser el evento actual y es posible acceder a él desde las acciones subsiguientes que forman parte de la ejecución hasta finalizar iniciada por ese evento.

Una transición que carece de un evento disparador explícito es lo que se denomina una transición de finalización (o transición sin disparador) y se dispara implícitamente al finalizar cualquier actividad interna del estado. Si un estado no posee actividad interna o estados anidados, entonces cuando se entra en el estado después de haber ejecutado cualquier acción de entrada se dispara inmediatamente una transición de finalización. Observe que una transición de finalización tiene que satisfacer su condición de guarda para poder dispararse. Si la condición de guarda es falsa al producirse la terminación, entonces el evento implícito de finalización se consume y la transición no se disparará más tarde aun cuando la condición de guarda pase a ser verdadera. (Este tipo de comportamiento se puede modelar, de otro modo, empleando un evento de cambio.)

Observe que todas las apariciones de un evento dentro de una máquina de estados tienen que tener la misma signatura.

Condición de guarda. La condición de guarda es una expresión booleana que se evalúa cuando se dispara una transición por haber manejado un evento, incluyendo los eventos implícitos de finalización que se producen en una transición de finalización. Si la máquina de estados está realizando un paso que se ejecuta hasta finalizar cuando se produce un evento, entonces se guarda el evento hasta que el paso ha finalizado y la máquina de estados está en reposo. En caso, contrario, se maneja el suceso inmediatamente. Si la expresión produce el valor verdadero al ser evaluada, la transición puede dispararse. Si la expresión toma el valor falso, entonces la transición no se dispara. Si no hay ninguna expresión que pueda dispararse, se ignora el evento. Esto no es un error. Es posible que múltiples transiciones que posean distintas condiciones de guarda sean disparadas por el mismo evento. Si se produce el evento, entonces se comprueban todas las condiciones de guarda. Si hay más de una condición de guarda que sea verdadera, sólo se dispara una transición. Si no se da una regla de prioridad, la elección de la transición que se dispara no es determinista.

Observe que la condición de guarda sólo se evalúa una vez, en el momento en que se maneja el evento. Si la condición produce el valor falso al ser evaluada y después se vuelve verdadera, la transición no se disparará mientras no se produzca otro evento y la condición sea verdadera en ese momento. Observe que una condición de guarda no es la forma correcta de monitorizar continuamente un valor. Para una situación como ésta debería emplearse un evento de cambio.

Si una transición no posee una condición de guarda, entonces la condición de guarda se trata como si fuera verdadera y la transición se activa si se produce un evento disparador. Si están habilitadas varias transiciones, sólo se dispara una de ellas. La selección puede no ser determinista.

Por comodidad, se puede descomponer una condición de guarda en una serie de condiciones de guarda más sencillas. De hecho, se pueden ramificar varias condiciones de guarda a partir de un único evento disparador o condición de guarda. Toda ruta que pase por el árbol representa una única transición disparada por el (único) evento disparador con una condición de guarda efectiva diferente, que es la conjunción (“and”) de las condiciones de guarda existentes a lo largo de su trayectoria. Todas las expresiones que haya a lo largo de la trayectoria se evaluarán antes de seleccionar la transición que se dispare. Una transición no puede dispararse parcialmente. De hecho, un conjunto de transiciones independientes puede compartir una parte de su descripción. La Figura 13.179 muestra un ejemplo.

Observe que los árboles de condiciones de guarda y la capacidad para ordenar transiciones que se pueden disparar son únicamente una cuestión de comodidad, porque se podría conseguir el mismo efecto mediante un conjunto de transiciones independientes, dotando a cada una de ellas de su propia condición de guarda disjunta.

Acción. Una transición puede contener una expresión que describe una acción. Se trata de una expresión para una computación procedimental que puede afectar al objeto que posee la máquina de estados (e indirectamente a otros objetos a los que tenga acceso). Esta expresión puede hacer uso de los parámetros del evento disparador, así como de los atributos y asociación del objeto que la posee. El evento disparador está disponible como evento actual durante todo el paso ejecutado hasta finalizar que ha sido iniciado por dicho evento, incluyendo los segmentos sin disparador y las acciones de entrada y de salida posteriores.

Una acción puede ser una secuencia de acción. Las acciones son atómicas, esto es, no se puede imponer su terminación externamente y tienen que ser ejecutadas por completo antes de que sea posible manejar ninguna otra acción o evento. Las acciones deberían tener una duración mínima, para evitar bloquear la máquina de estados. Los eventos que se reciben durante la ejecución de una acción se guardan hasta que finaliza la acción, momento en que son evaluados.

Ramificaciones. Por comodidad, varias transiciones que compartan el mismo evento disparador pero tengan diferentes condiciones de guarda se pueden agrupar en el modelo y en la notación para evitar la duplicación del disparador o de la parte común de la condición de guarda.

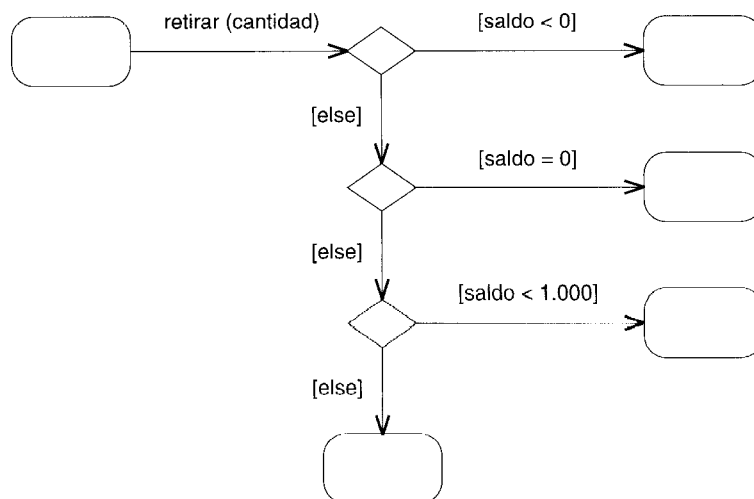


Figura 13.179 Árbol de condiciones de guarda

Esto no pasa de ser una forma cómoda de representación y no afecta a la semántica de las transiciones.

Véase bifurcación para más detalles de la representación y de la notación.

Notación

Las transiciones se representan mediante una flecha continua que va desde un estado (el de *origen*) a otro estado (el estado *destino*), etiquetadas con una cadena de transición. La Figura 13.180 muestra una transición entre dos estados y una que se ha fragmentado en segmentos.

Las transiciones internas se representan mediante una cadena de transición situada dentro de un símbolo de estado. Las cadenas de transición tienen el formato siguiente:

nombre :_{opt} nombre-evento_{opt} (lista-parámetros)_{opt} [condición-guarda]_{opt}
/ lista-acciones_{opt}

El *nombre* se puede utilizar para hacer referencia a la transición en las expresiones, especialmente para formar marcas de tiempo. Va seguido por dos puntos.

El *nombre-evento* da nombre a un evento y va seguido por sus parámetros. La lista de parámetros se puede omitir si no hay parámetros. El nombre del evento y la lista de parámetros se omiten en las transiciones de finalización. La lista-parámetros posee el formato siguiente:

nombre :tipo_{lista}

La *condición-guarda* es una expresión booleana escrita en términos de los parámetros del evento disparador y de los atributos y enlaces del objeto que describe la máquina de estados. La condición de guarda también puede implicar comprobaciones del estado de estados concurrentes de la máquina actual o estados indicados especialmente de algún objeto que se pueda alcanzar; entre los ejemplos se cuentan [**in** estado1] y [**not in** estado2]. Los nombres de estados pueden estar completamente calificados con los nombres de los estados anidados que los contienen, lo cual da lugar a nombres de ruta de la forma Estado1::Estado2::Estado3. Esto se puede utilizar si aparece el mismo nombre de estado en distintas regiones de estados compuestos dentro de la máquina global.



Figura 13.180 Transiciones

La *lista-acciones* es una expresión procedimental que se ejecuta siempre y cuando llegue a dispararse la transición. Puede estar escrita en términos de operaciones, atributos y enlaces del objeto poseedor y también empleando los parámetros del evento disparador. Entre las acciones se pueden incluir llamadas, envíos y otras clases de acciones. La lista-acciones puede contener más de una cláusula de acción separada por delimitadores formados por el signo de punto y coma:

acción_{lista}.

Ramificaciones. Una transición puede incluir un segmento con un evento disparador seguido por un árbol de estados de unión, que se dibujan como circulitos. Esto es equivalente a un conjunto de transiciones individuales, una para cada posible ruta a través del árbol, cuya condición de guarda es el “and” de todas las condiciones que se vayan encontrando a lo largo de la ruta. Sólo puede tener una acción el segmento final de la ruta.

Alternativamente, se puede dibujar un estado de unión como un rombo, si representa una ramificación o una reunificación. No hay diferencia de significados.

Discusión

Las transiciones representan un cambio atómico de un estado a otro, acompañado posiblemente por una acción atómica. Las transiciones no se pueden interrumpir. Las acciones de la transición deberían ser computaciones cortas, normalmente triviales, tales como sentencias de asignación y sencillos cálculos aritméticos.

transición abreviada

Notación que indica que una transición pasa directamente a un estado compuesto, pero se suprimen los detalles.

Véase también estado abreviado externo.

Notación

Una *abreviatura de estado* se representa mediante una pequeña línea vertical que se dibuja dentro de los límites del estado que la contiene (Figura 13.181). La abreviatura puede estar etiquetada con el nombre del estado, pero es frecuente que se omita cuando se suprimen los detalles. Indica que una transición está conectada a un estado interno suprimido. Las abreviaturas no se utilizan para transiciones a estados iniciales ni para salir de estados finales. Una abreviatura muestra que hay subestados adicionales en el modelo, pero faltan en el diagrama.

Un estado abreviado externo dentro de un estado de referencia a submáquina (Figura 13.91) hace referencia a un estado perteneciente a la definición de submáquina correspondiente (Figura 13.92). No se trata de un caso en que se hayan suprimido los detalles y es imprescindible incluir el nombre de la abreviatura.